

UPPSALA
UNIVERSITET

IT mDV 26 010
Degree project 30 credits
June, 2026

Bringing Bandits to Atlantis

Reinforcement Learning for Metaheuristic Selection in Constraint-Based Local Search

Simon Lalloo



Master's Programme in Computer Science



UPPSALA
UNIVERSITET

Bringing Bandits to Atlantis

Simon Lalloo

Abstract

This thesis aims to study, improve, and extend the constraint-based local search solver Atlantis. It investigates issues in the parallel performance of the solver, leading to the discovery of several critical bugs. The first of these bugs affected the probe acceptance rate in the local search process, and the second was an issue in the seeding of the random number generators when the solver was used with multiple threads. In the process of resolving these bugs it was also found that the use of some tracking statistics in the benchmarks negatively affected performance due to its use of shared memory, leading to much of the tracking being removed. Benchmarking now shows that parallelism does improve performance, but that cache contention limits how well this scales in practice.

The main work concerns studying the use of multi-armed bandits, a subset of reinforcement learning, for metaheuristic selection or tuning in constraint-based local search. The major difficulty is caused by constraints in the reward design, limiting which algorithms can be used, and eliminating the possibility of determining bounds on the algorithms' loss. A number of methods are implemented in Atlantis and benchmarked to very encouraging results. The most performant algorithms are found to be Thompson sampling and Upper Confidence Bound using Kullback-Leibler divergence. Although it is difficult to measure any exact degree of improvement, results show that on average the solver performance improves significantly while the amount of work for the user can be the same or even decrease.

Faculty of Science and Technology

Uppsala University, Uppsala

Supervisor: Frej Knutar Lewander Subject reader: María Andreína Francisco Rodríguez

Examiner: Lars-Henrik Eriksson

Contents

1 Introduction **1**

2 Background **2**
2.1 Combinatorial Problems . . . 2
2.2 Local Search . . . 3
2.3 Constraint-Based Local Search . . . 4
2.4 Multi-Armed Bandits . . . 5
2.5 Atlantis . . . 9

3 Parallel Performance Study **11**
3.1 Initial Cache Testing . . . 12
3.2 Probing for Bugs . . . 12
3.3 Random-Number Seeding . . . 13
3.4 Additional Improvements . . . 13
3.5 Results . . . 13
3.6 Cache Performance . . . 17

4 Bandit Design **19**
4.1 Extending the Solver . . . 19
4.2 Implementing the Algorithms . . . 20
4.3 Rewarding the Bandits . . . 20
4.4 Arming the Bandits . . . 22
4.5 Evaluating the Algorithms . . . 24

5 Experimental Results **25**
5.1 Benchmarking the Algorithms . . . 25
5.2 Using Tuned Annealing Schedules . . . 28
5.3 Pulling Arms . . . 31

6 Analysis **33**
6.1 Analysis of the Algorithms . . . 33
6.2 Analysis of Pull Counts . . . 34
6.3 Analysis of Tuned-Arms and Discussion of results . . . 35
6.4 The Default Annealing Schedule . . . 36

7 Conclusion **37**
7.1 Conclusion . . . 37
7.2 Future Work . . . 37

1 Introduction

Combinatorial problems are problems requiring an optimal assignment or arrangement to be found from a discrete set of possibilities [13]. Although not well-known by name, these problems appear in a vast range of real-world applications, from scheduling [23] and logistics [14] to resource allocation [2] and route planning [1]. Solving them efficiently remains an open challenge, and a wide variety of solving technologies have been developed in response [4, 6, 13, 25, 32]. These can be broadly divided into two groups [13]: model-based systematic methods offering useful theoretical guarantees, and stochastic methods trading these guarantees for performance that is often substantially better in practice. The first group contains technologies such as Boolean Satisfiability (SAT) solving [4], Satisfiability Modulo Theories (SMT) solving [6], and Constraint Programming (CP) [25]. The most well-known technology in the second group is (stochastic) Local Search (LS) [13], which moves through the search space in a stochastic manner, evaluating assignments through a set of metaheuristics.

Constraint-Based Local Search (CBLS) [32] is an attempt to design a solver in between the two groups, bringing the model-based abstraction of CP to the speed of LS, promising both efficiency and ease of use. Despite this, CBLS remains underdeveloped relative to other solving technologies [5], and current solvers leave significant performance on the table. This stems from the difficulty of abstracting the problem from local-search solver design, which is normally extremely problem-specific. The design of the search space resulting from different modeling constraints has been studied due to its importance in solver design [32], but the abstraction of many other parts remains unexplored, to the best of our knowledge.

In local search solver design, several important decisions still need to be made largely through testing different options and parameters to find one that works. Some research has been published looking into automating these choices with the help of reinforcement learning methods [3, 22, 33], but to the best of our knowledge, no such research has been directed at CBLS specifically. It is in CBLS where such automation is truly necessary, since these are the important factors limiting the performance of current CBLS-based general-purpose solvers, which now have to either use defaults, drastically limiting performance potential, or require the user to specify values, limiting how far the problem can be abstracted from the solver. This work is an attempt to address that gap.

This thesis amends and extends the CBLS solver Atlantis in two steps. The first is a thorough analysis of Atlantis's parallel execution, leading to the discovery and resolution of several critical bugs that severely limited performance, and investigating the limiting factors of multithreaded scaling. The second is the introduction of multi-armed bandit algorithms [18], a form of machine learning, to automate metaheuristic selection at runtime, replacing a default configuration with an online learning approach that adapts to each problem. The choice of metaheuristic is among the most impactful decisions in local search [25], yet in a general-purpose CBLS it is left to the user, undermining the abstraction that the technology is meant to add.

The thesis is structured as follows. Section 2 provides the necessary background on combinatorial problems, LS, CBLS, multi-armed bandits, and the Atlantis solver. Section 3 describes the parallel performance investigation and the resolution of the bugs uncovered during it. Section 4 presents the bandit-based metaheuristic selection framework, including the algorithms implemented and the challenges of reward design, and section 5 the results of benchmarking this extension. Section 6 offers a general analysis of the work and the benchmarks, and section 7 a final conclusion, summarizing all the work.

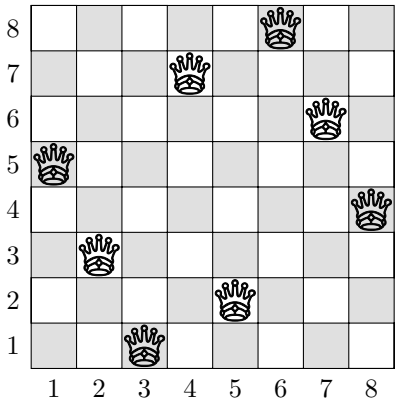


Figure 1: A chessboard depicting a solution to the n-queens problem for $n = 8$.

2 Background

This section explains the background necessary to understand this thesis. First, section 2.1 provides a deeper understanding of combinatorial problems and the difficulties inherent to solving them. Next, sections 2.2 and 2.3 go into more detail about the relevant solving technologies, discussing how they work and their relative benefits. Section 2.4 contains background on the multi-armed bandit problem and details various algorithms used to solve it. Finally, section 2.5 discusses the specific CBLS solver that was extended for the purposes of testing the theory developed in this work.

2.1 Combinatorial Problems

Combinatorial problems (or discrete problems) [25] are mathematical problems with a discrete set of solutions, commonly through variables being able to take only (certain) integer values. They are generally divided into two groups: satisfaction problems and optimization problems. Satisfaction problems are those asking for any solution satisfying certain constraints, while optimization problems ask for the solution optimizing the value of an objective function.

A common example of a combinatorial problem is the n-queens problem [19]. Given an $n \times n$ chessboard and n chess queens, this deceptively simple problem asks for an assignment of the queens to positions on the board such that none of the queens is threatened by any other. Since the problem provides no way to differentiate between the quality of different solutions, every valid solution is equivalent, meaning that the problem is a satisfaction problem. Figure 1 shows an example of a valid solution to an instance of the problem where $n = 8$.

As with all combinatorial problems, there are many ways to model the n-queens problem. Designing such a model requires identifying the variables and constraints in the problem. It is evident that some set of variables denoting the location of each queen on the chessboard is necessary. The specific values these variables should take are what the solver must determine. Chess queens can move any number of steps along their row, column, or diagonals, so the model needs the following constraints:

1. There must be exactly n queens in total.
2. Each queen must be assigned a position on the $n \times n$ chess board.

3. Multiple queens cannot be assigned to the same position.
4. No two queens can be on the same row.
5. No two queens can be in the same column.
6. No two queens can be on the same diagonal.

These constraints can be modeled in various ways. In general, constraints can be modeled *explicitly*, meaning that they are written directly, or *implicitly*, which means that the model is designed in such a way that the constraint will always hold, without having to be considered. In this example, constraint one could be modeled implicitly by simply having n location variables (or sets of location variables), automatically denoting exactly n queens. Using implicit constraints often simplifies models and makes finding solutions easier by reducing the search space, making them important to consider when modeling.

Mathematical optimization problems are normally solved through various methods based on the use of first- and second-order derivatives in order to find critical points. Such methods generally cannot be used with discrete problems however. Derivatives are defined only where a function is continuous, but discrete problems often do not come with continuous functions.

2.2 Local Search

Local search (LS) [13] is a nondeterministic solving technology for combinatorial satisfaction and optimization problems, which works by starting from some initial state and iteratively improving it in order to hopefully find a satisfying or optimal solution. Its name comes from the fact that in each step it considers only states close (local) to the current state (the *incumbent*), and picks one of these to move to. These nearby states often vary only in the value of a single variable, or have two variables swapped, or some other very minor difference. The set of states considered to be near a given state is called its *neighborhood*, and their structure is crucial in designing efficient LS solvers. Members of this neighborhood are chosen for consideration randomly in a process called *random probing*.

A *probe* consists of one of these neighboring solutions, usually picked at random [19]. The LS process uses heuristics to evaluate a probe and get some cost value, and a *metaheuristic* to determine if a probe should be accepted based on this cost and the cost of the incumbent [13]. In its simplest form, any improving neighbor would immediately be chosen as the new incumbent, with any worsening solution being ignored. One of the biggest challenges in LS however is the problem of *local optima*: points that are not optimal, but better than all their neighbors. A simple example of this is shown in figure 2, where the left cross marks the global optima, and the right cross marks a local optima. Greedy solutions such as the one just described have a very high risk finding a local optima, then getting stuck, since no direct improvements are possible. In the figure, this would happen if the solver happens to start anywhere along the right side of the "curve". The solution therefore has to get worse before it can get better. Finding a good balance between accepting worsening moves and moving towards optima is thus critical to the performance of an LS solver, making the metaheuristic one of the most important components of a solver.

One of the most common metaheuristics is simulated annealing [7]. It is named for the metallurgical process of annealing, wherein metal is heated and allowed to slowly cool to release stress. In simulated annealing, improving probes are always accepted, while worsening moves have a chance to be accepted based on the current *temperature*, with higher temperatures being likelier to accept worsening solutions. In each annealing round, the temperature starts with a high

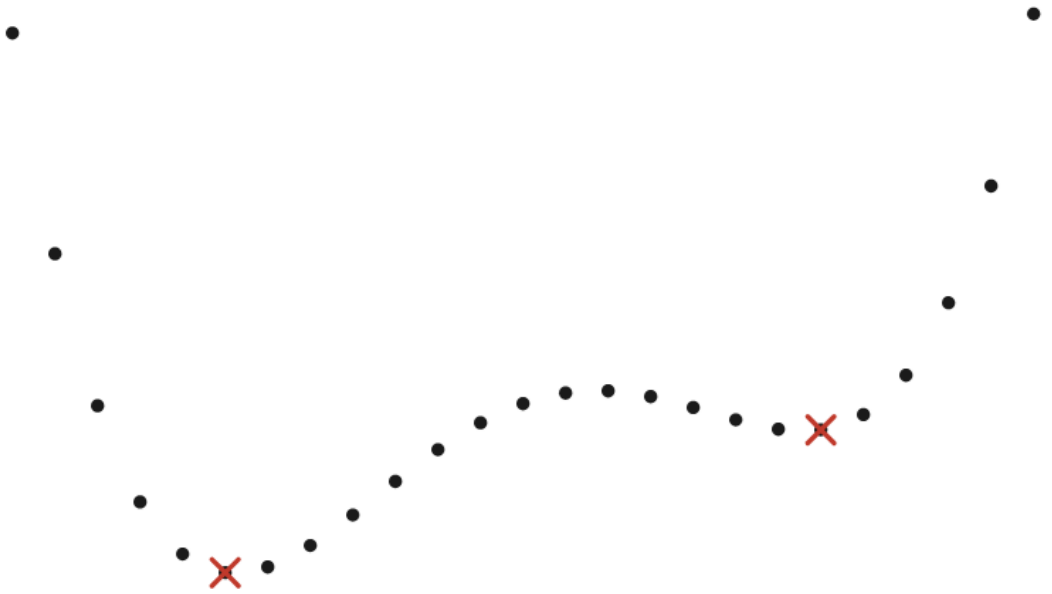


Figure 2: Let $y = f(x)$. Find the value for x minimizing y.

value and gradually cools down. This is meant to allow the solver be freer to make worsening moves early in a round, hopefully moving it out of any local optima, then gradually increasing the proportion of improving moves to bring the solution toward a new optima. Once the rate of accepted probes becomes too low, another round of heating can be applied, again raising the temperature, and the cycle restarts.

Considering the n-queens problem from an LS perspective, each state's neighborhood could be defined by the set of other states where one pair of queens have had their rows swapped. The cost of a state could be defined as the number of ways any queen could take any other queen, with the goal being to reach a cost of zero. Figure 3 shows what a probe could look like here. The complicating factor is that even though the problem is very simple, this would require a custom implementation, building the structures and creating the move and evaluation functions from scratch.

Unlike systematic search methods, LS does not consider the entire search space, and thus offers no guarantee of satisfaction or optimality. The trade-off made in losing this guarantee is that it can iterate very quickly, and is often able to find good solutions faster than systematic search technologies. Although the idea of local search is very general, implementations do not generalize well and bespoke solvers are usually required for every problem, making the method time-consuming to use in practice compared to solvers that can abstract the problem from the solver. This comes from the fact that neighborhoods, and thus probes, are very dependent on the individual problem.

2.3 Constraint-Based Local Search

Constraint-Based Local Search (CBLS) [32] is a solving technology combining local search with constraint programming (CP), which in turn is a solving technology abstracting the problem from

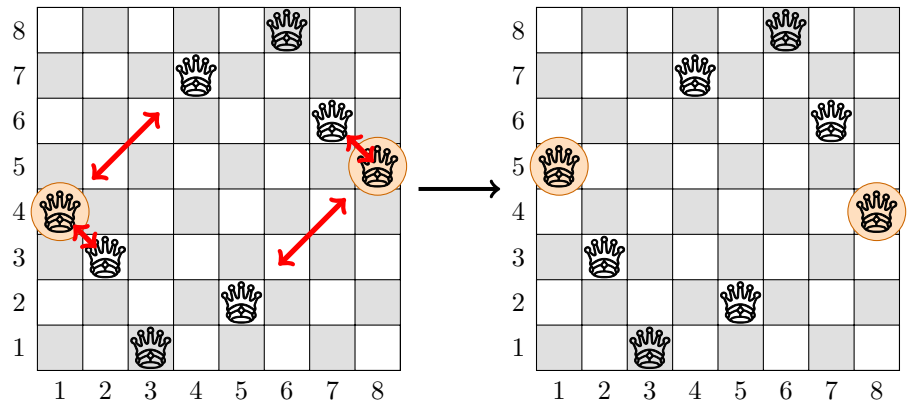


Figure 3: An example probe in the n-queens search. Here the left board (the incumbent) has four violations while the right board (probed state) has none, so the probe should be accepted.

the solver through the use of a model describing the constraints a solution must fulfill. These models are often written in high-level modeling languages such as MiniZinc [28]. Combining these technologies leads to a solver design with many of the same benefits as LS, but without the need to design a custom solver for every problem.

When solving the n-queens problem with CBLS we no longer need to design an entire solver, since just a model of the problem is sufficient. The CBLS solver would automatically convert this model into an LS problem, creating the structures and designing reasonable moves based on the declared constraints. The solving process can then make random probes in the local search style, with each probe causing the constraints to be iteratively updated and reevaluated [19].

A major challenge in CBLS solver design is adapting the very problem-specific LS technology to work in a general-purpose solver. Neighborhood design is closely tied to the implementation of various constraints, but the choice of parameters for the metaheuristic (often called *hyperparameters*) remains too instance-specific to be able to be reasonably derived from the model. This means that metaheuristic parameters still have to be specified by the user in order for the full performance of the solver to be unlocked. This is unfortunately counter to the simplicity and solver-abstraction the technology attempts to achieve.

2.4 Multi-Armed Bandits

Automated decision-making is a very well-studied problem [20]. It is a very broad topic, and is thus divided into many different areas based on the problems' characteristics. When the properties of the different alternatives are unknown but fixed, such that picking an option does not affect the others, this is called the Multi-Armed Bandit problem (MAB). The book *Bandit Algorithms* by Lattimore and Szepesvári [18] briefly covers its history and thoroughly describes the problem and a number of common algorithms used for it. The name multi-armed bandit comes from classical slot machines, which are sometimes known as one-armed bandits since they have one lever, or "arm", to pull and will almost inevitably take the player's money. When faced with several of these machines, the problem becomes figuring out which one has the best odds. Since the only way to evaluate the quality of an arm is to pull it, this naturally becomes a reinforcement learning problem. The problem is an old one, having been studied since the 1930s, and has applications in a wide variety of areas.

Reinforcement learning is one of the basic paradigms of machine learning. Sutton and Barto describe it as "learning what to do — how to map situations to actions — so as to maximize a numerical reward signal." [29]. This description, although simple, captures the very large field in one short sentence. The *reward* signal that they mention is what the algorithm uses to learn. In order for the algorithm to work, the value of the reward thus needs to correspond to the quality of the action, either directly or indirectly. If the learning happens in real time as the algorithm interacts with the environment, we say that the algorithm is *online*. Online algorithms generally work in some sort of *rounds*, wherein the algorithm chooses some action or set of actions and uses the ensuing reward to update its understanding of the environment and how the actions correspond to rewards.

In reinforcement learning there are two general types of actions which must be balanced: exploration and exploitation. *Exploration* describes actions taken with the intent of trying to find better actions, whereas *exploitation* is choosing the action with the highest expected reward. Achieving a good balance here is critical, since too much exploration leads to less time spent exploiting the best action, while too little exploration could lead to the optimal action never being discovered. The sequence of observed states and chosen actions is called the *history*, and the mapping from the set of histories to actions being learned is called the *policy*, usually denoted π.

The *regret* R relative to a policy π describes the difference between the expected reward from following π for n rounds and the actual reward obtained from those rounds. The regret is often measured relative to a set of policies Π, and is then the maximum regret relative to any policy in the set. The *suboptimality gap* or *immediate regret* is the difference in reward between the policy and optimal policy in a specific step.

Machine learning literature often refers to *actions* and *choices*, while MAB literature sometimes instead uses the terminology *arms* and *pulls*. Because the first set of terms is so overloaded, the latter alternative will be used throughout this thesis. When discussing multi-armed bandits, an algorithm's *horizon* is the total number of experiments it can perform. Many algorithms require the horizon to be known ahead of time. Algorithms without this requirement are called *anytime*.

We assume that the reader has a basic understanding of probabilities. For readers wanting a refresher, the necessary foundations are covered in chapter 2 of *Bandit Algorithms* by Lattimore and Szepesvári [18].

2.4.1 Explore-then-Commit

There are a number of different algorithms commonly used with bandit problems according to Lattimore and Szepesvári [18]. The most basic of these is called the explore-then-commit (ETC) algorithm, which simply plays each arm a fixed number of times (explore), then commits to whichever arm performed best, exploiting it until the algorithm ends. One issue with this method is that analysis of the regret requires knowledge of the suboptimality gap, which is rarely known in practice. Another issue is that it is not anytime, since any informed decision about the length of the exploration phase requires knowledge of the horizon. Setting the length of the exploration phase without the horizon risks either spending too much time exploring, resulting in a lack of exploitation, or starting to exploit without enough exploration time to make a good choice.

2.4.2 Thompson Sampling

Thompson sampling is not only a very old solution to the multi-armed bandit problem, having first been proposed in 1933 [30], but it is in fact the first algorithm proposed for it. Nevertheless, it has been proven effective and is still commonly used today, according to Lattimore and

Szepesvári [18]. Unlike many other algorithms which are structured around bounds, Thompson sampling is instead based on Bayesian statistics and the concept of posterior probabilities. Each arm is assigned some probability distribution. Whenever a choice needs to be made, the distributions are sampled and the arm with the largest sample is chosen. The posterior for that arm can then be updated based on the reward received. Over time the arms' posteriors will become more concentrated, decreasing probability of exploration as the true distributions are better approximated.

A crucial part of implementing Thompson sampling is the choice of distribution. This choice affects the results through the sampling and thus the way the arms are chosen, and affects the implementation since different distributions must be updated in different ways. An interesting detail is that the arms need not have the same distribution type, although in practice they usually do. When deciding which distribution to use, the most important part is knowing the rewards' supports (bounds), and whether the rewards are discrete or continuous. Knowing the approximate shape of their distributions is also useful, but the algorithm is quite flexible and can adapt even if the true distribution is not the same as the approximated one, albeit with reduced performance. It is very important here to differentiate between the distribution of rewards generated by an arm, and the conjugate prior distribution, which is the related distribution used to describe the actual (estimated) distribution.

As an example, consider a model with a Bernoulli-distributed reward $r \sim Be(p)$. The simple solution is to determine p by looking directly at the rewards. Using Bayes' theorem, we instead maintain another distribution that describes our belief about p's value. This is the distribution that Thompson sampling actually determines. When a reward is observed, we use Bayes' theorem to update this distribution.

$$\mathbb{P}(p \mid r) \propto \mathbb{P}(r \mid p) \times \mathbb{P}(p)$$

Using Bayes' terminology, the actual distribution is the likelihood function, while the prior and posterior use this related distribution. Calculating the closed form of such a related distribution is difficult, but often the solution is to use known pairs of related distributions called *conjugate distributions* with respect to the likelihood function. Continuing the example, the conjugate prior for a Bernoulli distribution is the Beta distribution. Many common distributions have conjugate priors, but for ones that do not approximation may be necessary.

2.4.3 Upper Confidence Bound

The Upper Confidence Bound (UCB) algorithm is more complex, but offers several advantages in return [18]. Like Thompson sampling, it is well-behaved even when there are more than two arms, and can be adapted to work without the horizon. The algorithm is built on the principle of *optimism in the face of uncertainty*, which states that we should assume that the environment is always as "nice" as is plausibly possible. From this principle it follows that for each arm we can find an *upper confidence bound* that overestimates the actual mean of each arm with a high probability. A non-optimal arm can then be played only if its bound is larger than that of the optimal arm, which is in turn larger than the optimal arm's mean. The result is that non-optimal arms are only chosen if there is a reasonable possibility of their actual mean being higher than that of the optimal arm, and the algorithm achieves sublinear regret.

There is no single convention for variables in the UCB literature [16, 17], but throughout this thesis the following convention is used. The current timestep is denoted by t and the number of arms by K. The measures $\hat{\mu}_i(t)$ and $T_i(t)$ are the empirical estimate of the return of arm i at time t, and the number of times it has been pulled at time t, respectively. A_t is the arm chosen to be pulled in timestep t.

The simplest anytime version of the algorithm is asymptotically optimal UCB [18]. An initial confidence bound is found by pulling each arm once. Thereafter, arms are chosen by picking the one with the highest upper confidence bound: $A_t = \arg\max_{i \in [K]} \gamma_i(t)$, where $\gamma_i(t)$ is the confidence bound function, defined as:

$$\gamma_i(t) = \left(\hat{\mu}_i(t-1) + \sqrt{\frac{2\log(1 + t\log^2(t))}{T_i(t-1)}} \right)$$

In 2015, Lattimore described a UCB variant called *Optimally Confident UCB* (OCUCB) [16], which improves on the basic algorithm by guaranteeing order-optimal problem-dependent and worst-case regret. The following year he expanded it to be anytime (OCUCB-n) [17]. The algorithm uses two tunable parameters $\eta > 1$ and $\rho \in [\frac{1}{2}, 1]$, and calculates the confidence bound as:

$$\gamma_i(t) = \hat{\mu}_i(t-1) + \sqrt{\frac{2\eta\log(B_i(t-1))}{T_i(t-1)}}$$

$$B_i(t-1) = \max \left\{ e,\ \log(t),\ t\log(t) \left(\sum_{j=1}^{K} \min\{T_i(t-1), T_j(t-1)^{\rho} T_i(t-1)^{1-\rho}\} \right)^{-1} \right\}$$

KL-UCB [10] is a variant of UCB designed by Garivier and Cappé that calculates the confidence bound with the help of the Kullback-Leibler divergence, a measure of how similar one distribution is to another. The algorithm works for any distribution, although the original paper provides regret bounds only for those in the exponential family. These bounds are not only promising in theory, but Garivier and Cappé also show numerical examples of their algorithm outperforming several other UCB variants. KL-UCB is especially noteworthy for being stable even for a short horizon. This stability is particularly important in cases where pulling an arm may take some time, which often causes horizons to be short in practice. The bound in KL-UCB is defined as:

$$\gamma_i(t) = \max \{ q \in [0,1] : T_i(t-1) \cdot d\left(\hat{u}_i(t-1), q\right) \leq \log(t) + c\log(\log(t)) \}$$

Where $d(p_1, p_2)$ is the KL-divergence between two distributions with parameters p_1 and p_2, respectively. The formula for this divergence varies based on the type of distribution, but for the Bernoulli distributions $\mathrm{Be}(p_1)$ and $\mathrm{Be}(p_2)$ we assume here, it is:

$$d(p_1, p_2) = \ln\left(\frac{1-p_1}{1-p_2}\right) + p_1 \cdot \ln\left(\frac{p_1(1-p_1)}{p_2(1-p_2)}\right)$$

Since q is not limited to integer values, finding the maximum value for q is a constrained optimization problem that can be solved using algorithms such as Newton's method.

Regardless of what UCB variant is used, these confidence bounds are used internally by the algorithm to evaluate the arms, but they are also useful in the calculation of larger bounds for the algorithm itself, i.e. upper and lower bounds on the total regret. Such bounds are unfortunately very dependent on the arms' distribution types. In cases where the underlying distribution type is unknown we can make assumptions, allowing us to decide what definitions to use for things like KL-divergence, but bounds may no longer hold.

The issue of deciding distributions and bounds is compounded when the reward distributions are not constant. Such cases are not unknown in the literature and are called *contextual bandits* [18].

Stochastic contextual bandits have rewards that depend on some external context, so this external context must be taken into account when calculating the UCB. There exists several UCB variants that do this, but they all assume either that the context is independent of what choices are made, or that the environment is an *adversarial bandit*, an environment that works actively to minimize the reward.

2.5 Atlantis

Rudbeck [26] wrote that the lost city of Atlantis was the root of all knowledge and culture. The CBLS-solver Atlantis[1] likewise provides a foundation on which various theoretical improvements to the technology can be developed and tested in practice. It provides a CBLS-based backend for the MiniZinc modeling language, implementing much of the theory described in sections 2.2 and 2.3. This section provides additional details on some parts of Atlantis that are especially relevant to this thesis.

An important part of LS, and thus CBLS, is how probes are evaluated. For each incumbent and probed state, Atlantis' heuristics create a cost-tuple, which describes its quality. The tuple consists of two integers: the *violation degree* and the *objective value*. The violation degree is a number describing how badly the constraints are violated. In the n-queens problem, this could describe the number of conflicts, as shown in figure 3. The goal is always to minimize this, reaching zero for a satisfying solution. The objective value on the other hand measures the quality of the solution, the value being optimized. In the example in figure 2, the objective value would thus correspond to the value of y.

Depending on the problem and how it is modeled, only one value in the cost-tuple might actually be used. The n-queens problem, for example, has no objective value to optimize as it is a satisfaction problem. When Atlantis is parsing a model, it converts explicit constraints into implicit constraints where possible. This can allow optimization problems to be solved as though they have fewer or even no constraints, simplifying (and thus speeding up) the solving process.

Every time a probe is made, the metaheuristic is given its cost and returns a decision, either accepting or rejecting the probe. Atlantis uses a variant of the simulated annealing metaheuristic [8], with a design that allows for multiple phases of heating and cooling at different rates before the metaheuristic needs to restart. This is controlled by an annealing schedule. As with many other parts of LS, good annealing schedules tend to be highly problem-specific. Unless a schedule is provided to the solver along with a problem instance, Atlantis will use a default schedule with parameters that have been determined to work well across a set of several problems.

Heating phases are used to gradually raise the temperature instead of simply assigning it some high starting value. The heating follows a geometric curve, with the temperature at round i of heating being $\tau_i = \tau \cdot \alpha_h^i$, where τ is the temperature at the start of the heating phase and $\alpha_h > 1$ is the heating rate. The temperature rises until the ratio of accepted worsening moves to probed worsening moves in a round exceeds a limit γ. In the cooling phases, the temperature follows a similar geometric curve, being described by $\tau_j = \tau_{\text{Hot}} \cdot \alpha_c^j$ at round j for some $0 < \alpha_c < 1$. The cooling phase continues until ϕ successive rounds show no improvement.

The metaheuristic alternates between these phases in a loop. If the solver completes a full set of heating and cooling phases without improving, the iteration is called *futile*. θ consecutive futile iterations indicate that the solver has reached a solution from which improvement is too difficult,

[1] Available at `https://github.com/astra-uu-se/atlantis`

Name	Hyperparameter	Value
Heating rate	α_h	1.2
Starting temperature	τ	1
Minimum uphill acceptance ratio	γ	0.75
Cooling rate	α_c	0.99
Successive futile cooling rounds threshold	ϕ	4
Maximum consecutive futile iterations	θ	5

Table 1: The default metaheuristic parameters in Atlantis.

and a complete restart is performed. The default values for these metaheuristic parameters are shown in table 1.

When evaluating the performance of a solver, a useful metric is the *optimality gap*, which is the cost-difference in percent between the result and the optimal solution. This is an easy way of describing how close a solution is to the optima, but it makes sense to use it only when the problem has no constraint violation. The number of constraint violations a solution has is zero, so the gap between any optimal and non-optimal solution is infinite. Problems with constraint violations are therefore evaluated using alternative measurements.

Local search is a nearly trivially parallelizable problem, and Atlantis has thus been designed to allow for parallel execution, running several independent LS threads simultaneously. It has support for several different levels of communication between the solver threads, ranging from allowing the threads to work completely independently to sharing solutions, with all threads jumping to the same solution whenever a new global best solution is found. Multiple inter-thread communication strategies were implemented in order to evaluate how closely the threads should work together to take as much advantage of the parallel execution as possible.

3 Parallel Performance Study

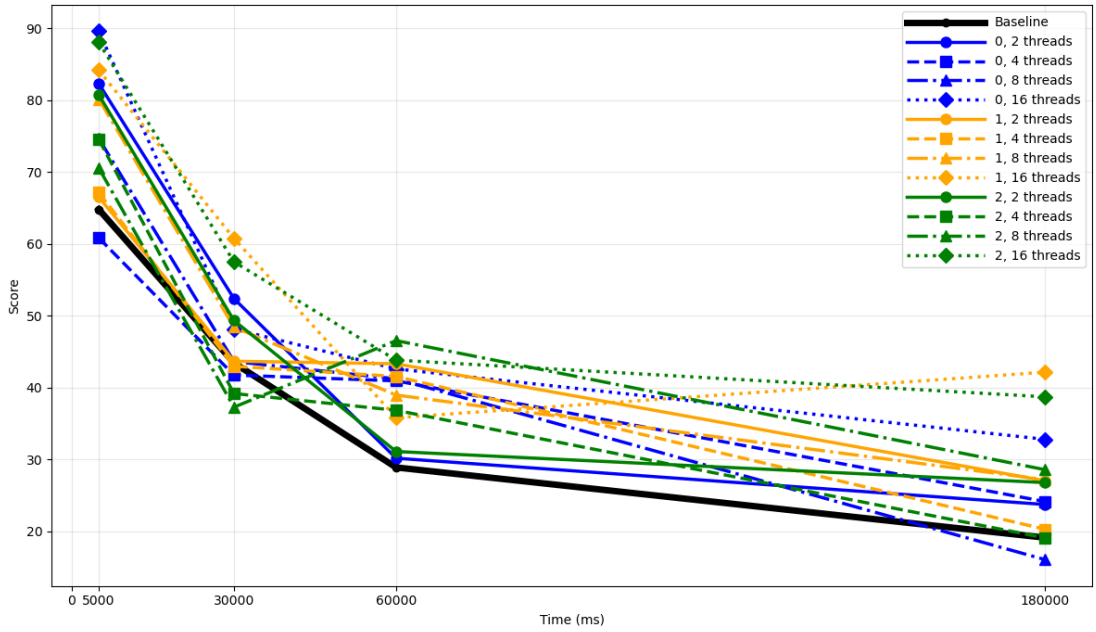


Figure 4: Mean normalized score (0-100) of TSP instances at different timeouts. Lower is better.

The project to parallelize Atlantis ended as most projects do: with an evaluation process. A set of benchmarks were run in order to measure the new performance. This consisted of three problems, each with a large number of problem instances. The problems were chosen for their relative simplicity as well as to provide good coverage of different parts of the solver, due to their respective models resulting in various neighborhoods being explored and in solutions with violation only, objective only, and the full cost with both violations and objective value. The problems used are:

- The n-queens problem, as described in section 2.1.
- The traveling salesperson problem (TSP) [9]: An optimization problem consisting of a traveling salesperson with a set of cities that need to be visited before returning, and the goal being to find the quickest route. This problem is modeled such that Atlantis is able to make all the constraints implicit, leaving it without possible violations.
- The knapsack problem [15]: an optimization problem in which the goal is to maximize the value of the contents of a knapsack of limited capacity, given some set of items with various weights and value.

The benchmark results were surprising in that solver performance appeared to decrease slightly as additional threads were added. Figure 4 shows an example of this, with the normalized mean score of the TSP tests using different thread counts and communication strategies. For the n-queens problem, the solver only consistently managed to solve instance $n = 16$, with the next largest tested instance ($n = 24$) being solved only a handful of times across all tested combinations using a three-minute timeout. Other solvers, such as Gecode [27] and Gurobi Optimizer [12], solve both these instances nearly instantly. The knapsack problem results all

found optima almost immediately, with solutions much better than the proven optima found by deterministic solvers, clearly indicating a bug, since this should be impossible. As a result of these performance issues, an investigation was launched into the solver's performance, looking at both overall performance as well as parallel performance specifically. Sections 3.1 through 3.4 describe this investigation, while sections 3.5 and 3.6 show the improved results achieved after resolving the issues this investigation highlighted.

3.1 Initial Cache Testing

Throughout the development process, cache contention was a source of concern as a potential limiting factor in performance improvements. CBLS can use large amounts of variables as a result of the problems' and instances' structures as well as the variables created by the solver when setting up the problem.

Cache performance was therefore evaluated using the Linux program `perf` [24] with a set of flags to count cache references and cache misses, for the L1 cache and last-level cache individually as well as for all caches together. Testing was carried out on a laptop with an Intel i5-1135G7, which has 4 physical cores and 8 logical cores, and three cache levels, the last of which is shared. The L1 data-cache is 192 KiB, the L1 instruction cache 128 KiB, the L2 cache 5 MiB, and the shared L3 cache 8 MiB. Tests were run with different numbers of logical threads and on several TSP and n-queens instances.

The results of the testing showed that cache miss rates were very low; less than 1% of L1 data-cache references were misses, and only a few percent of references to shared last-level caches. Although these rates were still enough that they may negatively affect performance, this could not explain the complete lack of improvement when multithreading, nor could it explain why overall performance was so much worse than other solvers.

3.2 Probing for Bugs

Having confirmed that the limiting factor was not cache contention, a deeper investigation was performed, analyzing the details of the solver's internal process. This consisted mainly of tracking more statistics in the solver and repeating benchmarks until an anomaly was found. It was eventually discovered that the probe acceptance rate was nearly 100%. One of the fundamental principles of LS solvers is that worsening probes should only sometimes be accepted, so this was a clear indicator of an issue in the probing process, since a significant portion of probes should statistically be worsening.

Following the annealer's process, it was found that the annealer was comparing probe results to a stored cost instead of fetching the incumbent's value each time. This stored value, however, was initialized along with the initial solution but never updated. Probes from an initial solution are very likely to have a better cost, and once a few improvements have been made further probes are almost guaranteed to be better than the initial solution. The result is that nearly every probe is accepted, causing the solver to perform something close to a random walk through the search space instead of the expected LS. A subset of the benchmarks, with the knapsack problem removed and a reduced number of TSP and n-queens instances, were repeated. Results were drastically improved, with n-queens instances two orders of magnitude larger being solved within the three-minute timeout used. For the 16-queens instance the solver was measured to perform roughly 15 times fewer probes before a solution was found.

3.3 Random-Number Seeding

The new run of benchmarks described in section 3.2 unfortunately still showed the same relative difference in multithreaded performance as before. With the cache contention hypothesis disproven, lock contention from the inter-thread communication was also quickly ruled out, and further tracking was added at a per-thread level. This revealed that threads were performing far more similarly than would be expected considering the highly random nature of the solver. Further analysis discovered that this was due to threads performing the same probes on the same initial solutions, stemming from their pseudo-random number generators being assigned the same seed. This had the consequence that every thread was performing the same work, causing any multithreading to only be duplication of work plus some slight overhead.

The necessity of using different seeds for each thread is obvious and had in fact been considered, but the code ensuring this had been lost during refactoring. Yet another iteration of benchmarking, with the same instances as above, finally resulted in the predicted outcome, with performance now being noticeably improved when multithreading.

3.4 Additional Improvements

Even after resolving these critical bugs, overall solver performance was worse than expected. Atlantis' benchmarking structures are made using the framework Google Benchmark [11], with statistics being recorded whenever the solver finds a new best solution. Certain statistics, such as solving time, are tracked automatically, but most need to be added manually. This needs to be done by registering them in the benchmark's setup method, then injecting a function into the solver, which the solver in turn has to call to update the statistics. In Atlantis specifically, this is done with the function `onSolution`, which as the name suggests is called by the solver whenever a new good solution is found.

In order to achieve more detailed tracking, a second function, which could be called to update statistics much more regularly, had to be injected into the solver. This caused a significant increase in shared memory operations, adding overhead and reducing the number of probes the solver can perform. Since the amount of overhead here is directly tied to the number of threads, this was severely limiting the performance improvements from multithreading. Removing much of this tracking finally lead to significant performance gains from multithreading, at the cost of no longer being able to track many interesting solver statistics.

A closer look at the knapsack problem results revealed that the solver was maximizing value by simply packing everything, disregarding the weight limit. This was found to stem from a bug in the implementation of one of the constraints. The broken constraint was not used for the other tested problems, hence the knapsack problem being the only one to produce incorrect solutions. The constraint could subsequently be fixed, thereby allowing problems using it to be correctly solved.

3.5 Results

A full suite of benchmarks was repeated to measure the new performance and evaluate the quality of the final parallelization solution. The benchmarks were again partially changed, with the number of instances of each of the previously mentioned problems being further limited in the interest of time. To ensure different problem sizes and difficulties were properly tested, at least five instances of each problem were kept. In addition to the previous problem set, the knapsack problem was brought back and a new problem was added. TSP with time windows

(TSPTW) [21] is an extension of TSP, in which each city has an added constraint restricting when it can be visited. The added complication changes the problem from one of pure optimization to one requiring both constraint satisfaction and objective optimization.

Performance benchmarking was carried out on the Pelle HPC cluster provided by UPPMAX (Uppsala Multidisciplinary Center for Advanced Computational Science) [31]. Each test was limited to a single node, featuring an AMD EPYC 9454P processor with 48 physical and 96 logical cores running at 2.75 GHz and boosting up to 3.8 GHz. The L1 data and instruction caches are 1.5 MiB each, the L2 cache is 48 MiB, and the shared L3 cache 256 MiB.

Figure 5 shows the mean optimality gap over time for the TSP tests. Each line shows the mean optimality gap of all the runs of all the instances of the problem, for one combination of search strategy and thread count. As expected, running multiple threads in parallel does indeed improve performance, with the single-threaded baseline being outperformed by every multithreaded line, regardless of search strategy. We can also see that increased thread count correlates strongly with improved performance. When it comes to search strategies, they appear to have only a small impact on results, but the cost-sharing search is the worst overall, with beam-search having the best performance, especially for timeouts of less than a minute.

Figure 6 shows the percent of instances solved in the n-queens tests. Here we again see a clear improvement correlating to thread count, but unlike in figure 5 there is also a distinct difference between the communication strategies, with the fully parallel search being superior to the other two, at least for more timeouts longer than 60 seconds. Cost-sharing search still seems to underperform, barely beating the baseline, especially for shorter timeouts. The way the various lines cross each other back and forth suggests a high level of natural variance, indicating that it may be good to repeat the tests with more than three runs per instance.

Results for the TSPTW are more complex since the problem has both (possible) constraint violations as well as an objective value to minimize. Figure 7 shows these results. Both the violation and objective value have been normalized into a range of 0 to 100 using the lower and upper bounds on the measured results for each instance. Their combined cost value is determined according to the following function:

$$\text{cost} = \begin{cases} \text{violation} + 100 & \text{if violation} > 0 \\ \text{objective value} & \text{otherwise} \end{cases}$$

This creates a combined cost that is above 100 for any violating solution, and where lower results are always better. The plot shows the average combined costs of all the runs of every instance, for each parameter combination. Multithreading continues to be clearly beneficial, with beam-search continuing to be the top performer.

Lastly, figure 8 shows the mean optimality gap for the knapsack instances. These results show far less improvement over time, but compensate by very clearly showing the benefit of multithreading as well as the difference between communication strategies. The very straight lines are likely caused by the solver getting stuck in local optima nearly immediately, causing it to have a chance of finding improvements only when it performs a full restart. The cost-sharing search is significantly worse than the baseline here, and unlike for the other problems the choice of search strategy appears to have a larger effect than thread count.

The speedups across the various problems and strategies may seem small at first glance; for the TSP tests the average optimality gap decreases by only a few percentage points when running on eight threads compared to one. It is therefore important to consider that the difficulty of

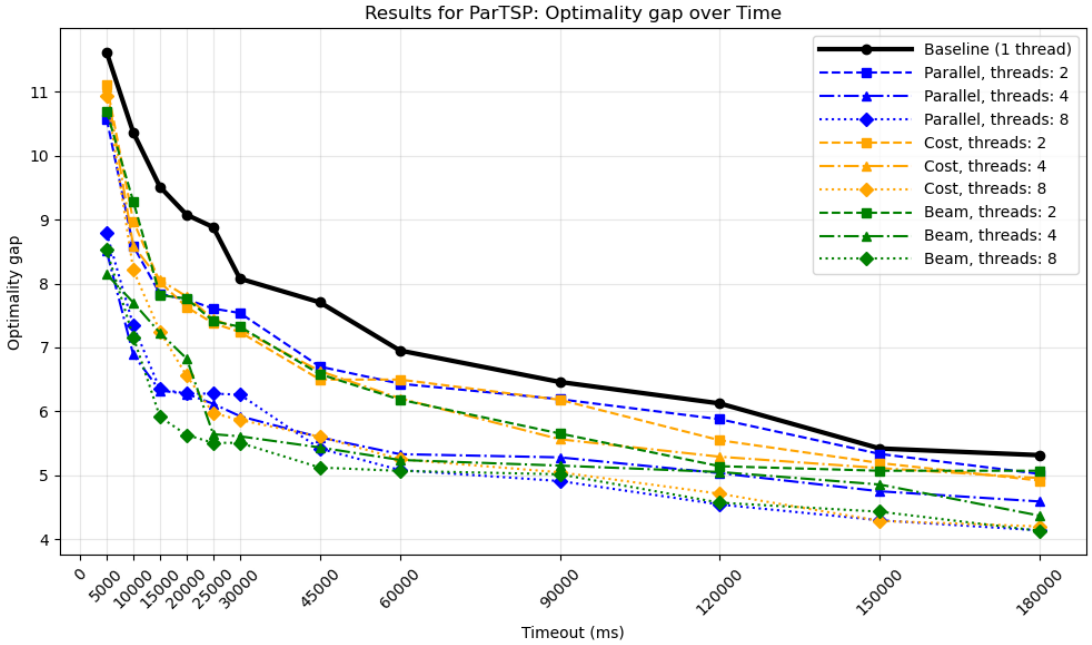


Figure 5: Mean optimality gap of TSP instances at different timeouts. Lower is better.

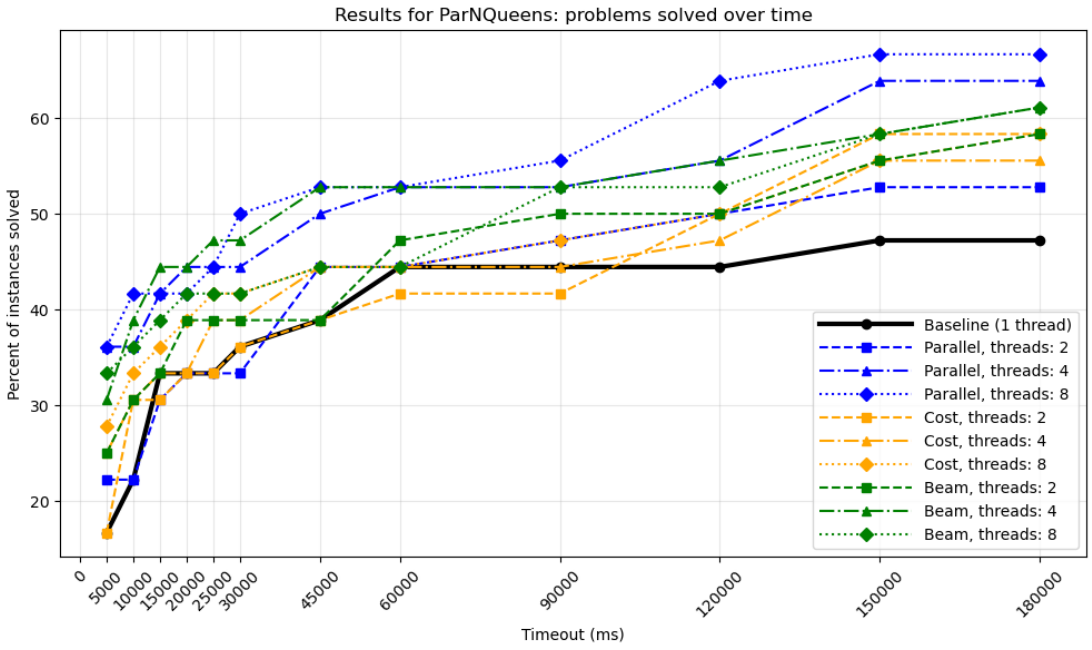


Figure 6: Percent of solved n-queens instances at different timeouts. Higher is better.

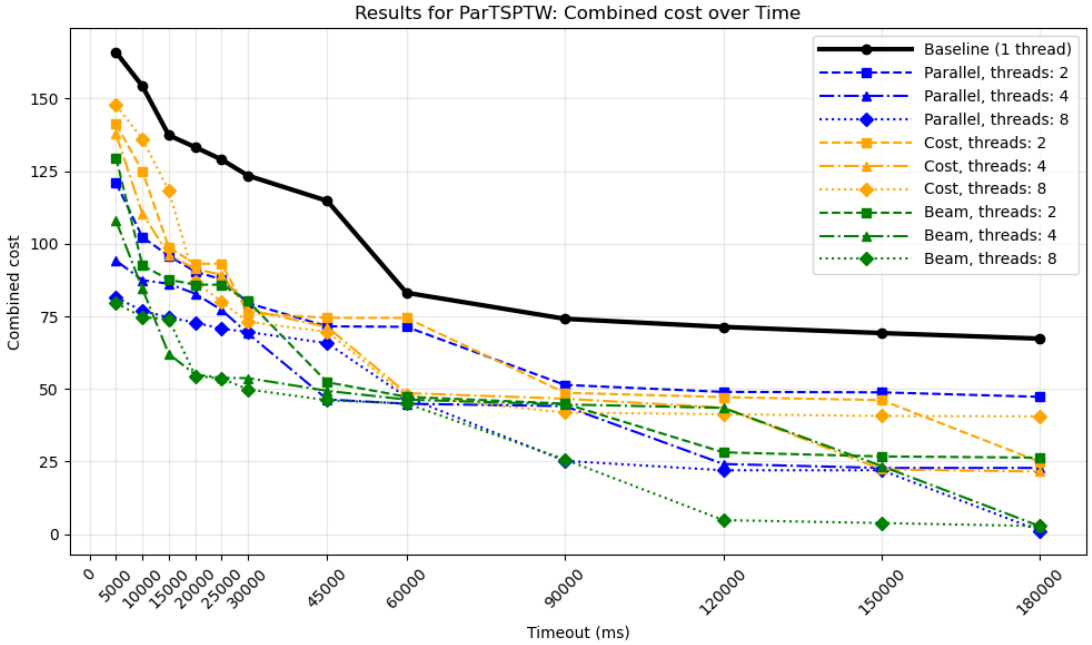


Figure 7: Mean combined cost of TSPTW instances at different timeouts. Lower is better.

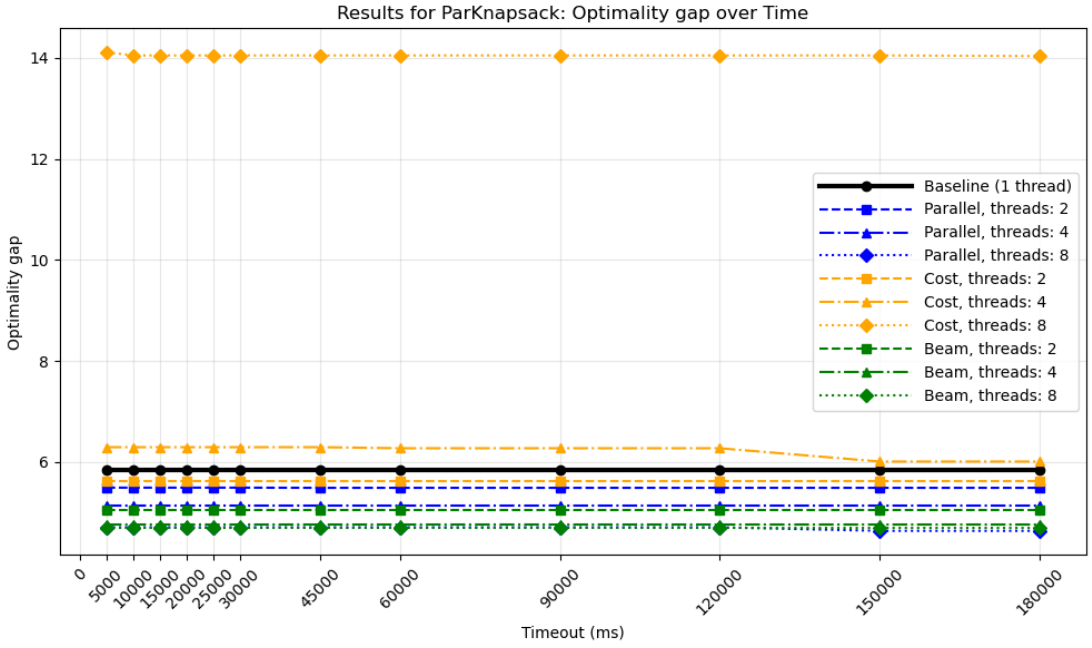


Figure 8: Mean optimality gap of Knapsack instances at different timeouts. Lower is better.

Thread count	Cache refs	Miss rate	L1 miss rate	LL miss rate
1 thread	9.3	33.69%	0.73%	9.53%
2 threads	47.7	23.99%	0.89%	18.17%
4 threads	96.3	11.41%	1.10%	6.58%

Table 2: The cache test results with a TSP instance and different thread counts.

combinatorial problems is rarely linear, but rather increases rapidly the closer the solver comes to the optima. These last few percent optimality might in fact require four or eight times more work. An unfortunate effect of this is that it makes it difficult to analyze how well the solver actually scales with thread count. A workaround could be to compare the results of n-threaded tests with single-threaded tests running n times as long, but the benefits of this are limited as it would work only for the fully parallel communication strategy.

Looking at the overall results, it is clear that multithreading is beneficial to solver performance, although not by how much. This aligns much more with the hypothesized results. In the original benchmarks multithreading reduced performance, and overall performance was extremely limited. Taking the n-queens problem as an example, the original tests were unable to solve the problem for $n = 24$, while the fixed version regularly solves the problem with $n = 1024$, albeit mostly when multithreading. A caveat here is that the hardware used in the original benchmarks was not as powerful as UPPMAX, but this difference is nowhere near enough to account for any significant portion of the results.

A somewhat surprising result was the overall fairly small difference between the different communication methods. These had been theorized to have significant effects on performance, especially when comparing fully parallel search and beam-search. The results show that they in fact perform similarly to each other, except on the knapsack problem. The cost-sharing search was the worst overall, which was expected, but even here the difference between it and the other methods was small and it occasionally outperforms other methods with the same number of threads. Its bad performance is likely due to one good thread causing the other threads to become "stuck", unable to accept improving probes and essentially reducing it to extra slow random search. When the one good thread then restarts, it causes the same problem for itself, since its new state will almost certainly be far from the best known values. Overall, beam-search seems to be the best method for short timeouts of less than $45s$, while for longer timeouts the broader search space coverage of the fully parallel search causes it to often outperform beam-search.

3.6 Cache Performance

With the solver's performance no longer limited by critical bugs, the previous cache testing results were no longer applicable. Another set of cache tests was therefore carried out to determine if cache contention was indeed an issue. These tests consisted of three runs each of an instance of TSP and an instance of the n-queens problem, with timeouts of up to 30 seconds and using the same hardware as in the first cache test, described in section 3.1. Because the amount of saved values in the solver should not increase once the first solution has been submitted to the shared control structure, longer timeouts were not judged necessary.

Tables 2 and 3 show the results of these tests for 1, 2, and 4 threads. The total number of cache references is listed in millions. These results show that although multithreading may lower the overall miss rate, the increase in total number of cache references indicates that the number of misses per thread still increase.

Thread count	Cache refs	Miss rate	L1 miss rate	LL miss rate
1 thread	29.5	16.50%	9.82%	9.82%
2 threads	211.0	30.53%	23.32%	29.16%
4 threads	317.9	14.95%	23.63%	14.68%

Table 3: The cache test results with an n-queens instance and different thread counts.

For TSP, this increased miss rate is monotonic in the L1 data-cache across all tests, but the shared last-level cache offers more varying results, with both miss rate at 4 threads being lower than at 1 thread, although the number of misses, both overall and per thread, still increases. The n-queens test shows more drastic results, with the overall miss rate being lower than for TSP, but the L1 data-cache miss rate being far higher. Interestingly, the miss rate more than doubles between one and two threads, but appears consistent between two threads and four. The last-level miss rate again increases when adding a second thread but decreases when adding another two threads.

Cache contention appears to have significant effect on the solver's multithreaded performance. For the traveling salesperson problem, although overall results improve when multithreading, the increase in total number of probes performed across all threads increases at a rate well below linear. When it comes to the n-queens problem the difference is even larger, as expected by the cache results, especially from the L1 cache. The increase in cache reference count compared to thread count is here even larger. As might be expected after seeing the high miss rate, the solver is unable to saturate its threads, with the time spent waiting for cache fetches causing utilization of its assigned logical processors to only be around $60 - 80\%$.

4 Bandit Design

As stated in section 2.3, the choice of metaheuristics and hyperparameters is critical to the performance of a CBLS solver, and yet it is often left to some default that was determined to be "good enough". If something better is desired, this is left up to the user to provide. This may work well for the local search solvers inspiring CBLS design, but here the solver sets up the problem automatically from the model. This section attempts to bring the same abstraction to the choice of metaheuristics, improving solver performance without the user having to concern themselves.

Whenever the metaheuristic's schedule causes the solver to perform a restart, the entire metaheuristic is reinitialized and nearly all the solver's internal structures are reset. Each such iteration is therefore independent, and the outcome of one does not affect the outcome of another. This makes it easy to replace the metaheuristic with a new one at this point. We can see that this decision problem matches the requirements for the multi-armed bandit problem. Pulling an arm in this scenario is thus equivalent to choosing a metaheuristic and letting the solver run until it decides to restart again.

We can further analyze how Atlantis works in order to determine the formal requirements on MAB algorithms for this problem. Since the time it takes to pull different arms can vary drastically and the solver timeout should be abstracted from the solving process, the number of pulls that can be made is unknown, necessitating an anytime algorithm. The algorithm must work with unknown optimality gaps, since nothing is known about problem or instance optima ahead of time. The algorithm only needs to support a finite number of arms, but also cannot be limited to just two.

4.1 Extending the Solver

The bandit algorithm is implemented through an interface called `ArmSelector`. This interface has multiple implementations, allowing the different algorithms to be chosen at runtime through the use of a flag. Each algorithm implements the functions `chooseArm` and `recordArmStats`. At the start of each solver cycle, the main loop calls the prior and is provided with a metaheuristic to use. Once the metaheuristic determines that a restart is necessary, the latter function is used to record the results of the pull, and the cycle restarts. The arms that the algorithm should consider can be provided as an input file in the json format in the same way as metaheuristic parameter sets. If this input contains more than one parameter set, they will automatically be treated as arms in a MAB problem.

Regardless of the algorithm chosen, the selector is a shared structure owned by the thread controller, allowing the MAB algorithm to work with any number of threads. Although nearly all work performed in the selector thus needs to happen behind a mutex lock, this does not have any significant effect on overall performance; arm recording and selection time are both in the order of tens of microseconds while pulling an arm often takes several seconds. This makes arm recording and selection a negligible fraction of the time an arm takes to run. The risk of lock contention is similarly extremely small, except when the solver starts and all threads need to choose arms at the same time. The implemented bandit algorithms all make the first at least k pulls be selected nearly immediately, where k is the number of arms, in order to ensure some baseline for all the arms is known before the more advanced calculations can be used.

4.2 Implementing the Algorithms

Explore-then-commit is the easiest bandit algorithm to implement, and the one with the smallest ability to adapt to variations in performance. In this case its largest flaw is that it is not an anytime algorithm. In order to get a reasonable balance between exploration and exploitation the algorithm needs to know when to stop exploring and start exploiting. The only way to make such a decision is with the use of a priori knowledge of at least roughly how large the horizon is. Because this is so strongly problem- and instance-dependent, this is not possible. A workaround could be to dynamically determine some mean pull time and use the timeout to pick this limit, but that would require significant changes to the solver and would result in an algorithm closer to the UCB family. A limit of four pulls per arm was instead chosen based on some limited performance testing with TSP, but as the results will show it is impossible to pick a value that works well on general problems.

The first of the more advanced bandit algorithms to be implemented was Thompson sampling. This algorithm was chosen due to its relative simplicity compared to many other algorithms. The crucial detail in the Thompson sampling algorithm is the choice of distribution. This choice is related to the expected format of the reward. Critically, the type of distribution used in the algorithm does not have to exactly match the actual reward distributions. Two distributions were considered here: the Bernoulli distribution, for binary rewards, and the Poisson distribution, for discrete non-binary rewards.

The first reward implemented was based on the number of improving moves the metaheuristic recorded. Since this is a non-negative integer value, it works well with the Poisson distribution and its conjugate prior, the gamma distribution. The reward was later changed to a binary value, so the Bernoulli distribution was implemented as well, with both alternatives being benchmarked. The Bernoulli distribution version of Thompson sampling can be expected to perform better, since the reward takes the form of a binary value, but the Poisson distribution seemed to work very well in initial testing, which is why it was also benchmarked. The two implementations are nearly identical, with the only difference being how the parameters α and β are updated with the reward, and which distribution is sampled when choosing arms.

Three variants of UCB were implemented: the basic asymptotically optimal UCB algorithm, KL-UCB, and OCUCB. The asymptotically optimal UCB algorithm was implemented as described in section 2.4.3. The OCUCB algorithm has two tunable parameters, η and ρ. They were set to 1.1 and 0.5; values that were determined to work well. For the KL-UCB implementation, the Bernoulli distribution was chosen due to the use of binary rewards. Newton's method is used to solve the optimization problem it contains, with the limit being set to five steps. This limit was found to provide good optimization results while limiting how long this internal solver can take. The optimization problem needs to be solved once for each arm every time the solver picks an arm to pull, so efficiency is important here. If five Newton steps do not find the optima, then the value will at least be close enough to it so as to make very little difference in the confidence bound.

4.3 Rewarding the Bandits

One of the biggest difficulties in reinforcement learning is designing good rewards. The reward is the only part of the solver that the RL algorithm can see; the feedback it uses to learn what to do. The reward thus needs to accurately reflect the result, being a measurement of not just success or failure, but preferably also of the degree of success. The algorithms themselves can often be taken almost directly from the literature, but reward design is based on the application. The

structure of the reward can also affect which algorithms can actually be used for an application. Somehow finding a single value that reflects the performance of an entire CBLS solver is no small challenge.

In this case, there are a few different values that could be used to make the rewards. We need a number measuring the quality of a solution, and through the structure of the CBLS solver we actually have one: the objective value. This cannot be directly used as a reward, however, since many problems have constraints with some violation degree that must be satisfied, and satisfaction problems like the n-queens problem have no objective value to look at. Any reward design using the objective value must then also be able to take into account the violation degree, which is where the complications start to compound. When directly comparing the quality of two solutions this is easy; the two values can just be compared lexicographically, prioritizing the violation degree. Such a solution is not possible for the reward however, since the bandit algorithms require a single value to operate on.

Combining these two values into one would need to be done in a way that ensures any non-violating solution produces a better reward than a violating one, and without entirely throwing off the scale or balance of the rewards in general. The most intuitive way to combine the two values would be to scale the violation degree by the largest possible objective value. This is infeasible in practice however, because the solver handles both these values as 64-bit integers, so combining them would require the use of 128-bit integers, which are not widely supported. Limiting the solver to smaller values such as 32-bit integers does not work either, as it is quite possible for the maximum cost or violation degrees to be reached in practice, even with these large values. Even if such a solution could be implemented, whether with 64 or 128 bits, the extreme difference in magnitude between different rewards would likely cause the bandit algorithms not to perform well.

Another useful value to consider is the number of improvements an arm makes from its initial random assignment. It is reasonable to say that an arm making more improvements is likely better, but this is not guaranteed. Since the initial solution is random, one arm might start from a much better position than another. This number also does not account for the size of the improvements, meaning that many very small improvements would score higher than a few large improvements, even if the latter resulted in a better objective value. On the other hand, this single number can mostly evaluate arms regardless of whether results have an objective violation, an objective value, or some combination thereof, thus making it easy to use as a reward.

A challenge with applying the bandit problem to CBLS is that the majority of values that we might want to use in the reward are highly problem- and instance-dependent. This actually presents multiple difficulties. Firstly, many algorithms are designed to work with either binary or normalized (or at least bounded) rewards. Looking at numbers such as the objective value or the number of improvements made, the only reasonable bounds that can be applied are those caused by implementation details, i.e. the types used to hold the values. These are not good bounds in practice, since for any given instance the values will likely be within just a fraction of this enormous range. Secondly, since there are more than one number that could reasonably be used in evaluating an arm's performance, we might want to combine them in some manner. Because there is no guarantee that the numbers are anywhere near the same order of magnitude however, it can be very difficult to balance the numbers in a way that allows combining them without risk of one completely overshadowing another.

One way to solve the issue of somehow normalizing numbers is with sigmoid functions, which are generally monotonic functions $f : \mathbb{R} \to [0, 1]$, and thus allow normalizing values without knowing

their bounds. Unfortunately just applying a sigmoid function is not enough to overcome this obstacle. When a sigmoid function's argument is far from its mean, the result will be very close to either zero or one. This becomes an issue since the number of improvements an arm makes, for example, is often in the hundreds or thousands. As an example, consider a simple sigmoid function $f(x) = (1 + e^{-x})^{-1}$. Here $f(100)$ and $f(1000)$ will differ by far less than machine epsilon[2], making the comparison impossible. Other sigmoid functions exist that behave better, but without some form of shifting the mean and scaling the values, they all face this issue to some degree, bringing us back to the same complication as before.

Finding some mean value to shift by requires having some values to average, which requires pulling the arms. Because the arms take different, instance-dependent, and nondeterministic amounts of time to pull, finding good such averages is not easy. It also presents a major issue, in that this mean must either be fixed at some point before the actual algorithm starts or the value needs to change dynamically throughout the solving process. An example can easily show that fixing the mean will not work in practice. If the three arms in a minimization problem have average objective values of 10, 100, and 1000, their mean would be 370, causing the solver to treat the first two arms as equally good, since both outperform the mean, even though the first arm is clearly much better.

One way to design the reward and be able to use these numbers is thus a dynamic approach, continually updating the mean or bounds, and using these to provide rewards. Although such approaches can work in practice, they have the enormous downside of effectively causing reward distribution to change over time. The bandit algorithms described in section 2.4 have all been designed to assume constant distributions of rewards from each arm. Dynamic distributions will not necessarily cause the algorithms to stop working, but they do mean that the bounds the algorithms provide are no longer applicable, limiting this thesis to purely experimental results, and requiring far more practical testing.

Several reward designs were considered, but thorough benchmarking was performed only with the one that worked best in preliminary testing, which was a dynamic design awarding points to pulls that performed well compared to a rolling average. The average is based on the mean violation degree and objective value of the n most recently recorded pulls, with $n = 10$ found to work well in practice. When a new pull is recorded, it is rewarded with a point if its best solution is better than this average. This reward design does not work with ETC however (unless an initialization phase before the regular exploration phase were to be added), so this method is limited to still using the number of improving moves made as the reward.

4.4 Arming the Bandits

Multi-armed bandit problems are all about figuring out which arm is the most promising. In the problem's namesake setting, the set of available arms is just the preexisting set of slot machines in the casino. Having defined our figurative casino, we thus need to populate it with slot machines. In this application the definition of arms is more difficult, since each arm is just a set of metaheuristic parameters. Many of these parameters use decimal numbers, meaning that the number of possible arms approaches infinity. Before starting the solver, the subset of arms to be considered must therefore be determined.

Although Atlantis has a default set of metaheuristic parameters, it also allows a custom set of parameters (called the annealing schedule) to be supplied by the user when starting the solver. This structure was extended to allow a list of multiple schedules.

[2]Machine epsilon ε is 2^{-53} for a double precision number in C++.

Meta-parameter	Arm 1	Arm 2	Arm 3	Arm 4
α_h	1.2	1.5	1.2	1.5
τ	1	1	1	1
γ	0.75	0.9	0.3	0.75
α_c	0.99	0.99	0.99	0.96
ϕ	4	4	6	5
θ	5	5	5	5

Table 4: The metaheuristic parameters of the first set of four arms, consisting of variations of the default arm.

Meta-parameter	Arm 1	Arm 2	Arm 3	Arm 4
α_h	1.2	1.5	1.5	1.2
τ	1	1	1	1
γ	0.75	0.3	0.3	0.2
α_c	0.99	0.9	0.8	0.99
ϕ	4	6	6	6
θ	5	5	5	5

Table 5: The metaheuristic parameters of the second set of four arms, consisting of the default arm and three arms tuned to the tested problems.

The default parameters were chosen due to their good overall performance across various problems, as determined by Flippo [8]. It is important to note however that this work was done several years ago in 2022, and the tests have not been repeated since, while Atlantis has had many updates causing changes in its performance. Determining whether these parameters are still optimal (or if they are even good anymore) is outside the scope of this thesis. Results of many informal tests performed during the development process indicate that some of them are in fact quite bad, with the tuned arms all reducing the heating acceptance ratio γ by more then 50%, for example, and that change alone seems to drastically improve performance across all problems.

Two sets of arms were created to be used for testing throughout the development process. The first of these is a set of variations of the default schedule. These have not been designed to be particularly good or bad, but rather to be quite similar to each other, so as to better evaluate the algorithms. It consists of the following four arms:

- The default schedule as described in table 1.
- An arm with a higher heating rate and higher acceptance ratio limit on the heating phase.
- An arm with a longer cooling phase and lower lower acceptance ratio limit on the heating phase.
- An arm with higher rates of both heating and cooling, along with slightly extended cooling phase.

Table 4 describes the specific parameter values used in the arms.

A second set of arms was created by determining good parameter sets for each problem[3]. These are shown in table 5, where arm 1 is the default parameter set, arm 2 the best discovered parameters for TSP and TSPTW, arm 3 the best parameters for n-queens, and arm 4 the best

for the knapsack problem. The hypothesis is that since these parameter sets are the best (known) parameter sets for each of the problems, the MAB algorithms should be able to pick the "correct" arm when running the benchmarks, thus resulting in performance approaching that of using only the tuned parameter set for that problem. The benefit is that the user does not have to change parameters when changing problems, as this is handled by the MAB algorithm. This approach does not generalize perfectly though, since these parameter sets may not be particularly good for problems other than those used in the benchmarks, but it is reasonable to assume that since the benchmark problems have been chosen for their good coverage of different constraints and neighborhoods in the solver, other problems should still perform well with at least one of these arms. This approach comes with another benefit in the testing process, in the fact that we can know that different arms are good for different problems, which should reduce risked of results being skewed by lack of exploration.

4.5 Evaluating the Algorithms

Having implemented multiple algorithms to solve the same problem, and chosen some sort of reward, it becomes necessary to find a way to evaluate them. Although this may at first seem easy, it is unfortunately a somewhat complicated task. Since the objective of this work is to make the solver perform better, the most straightforward measurement is just overall performance: given the same instances and arms, which algorithm gives the best objective values? The issue with such a measurement is that it does not work well with the exploration/exploitation dynamic, favoring any algorithm that spends more time exploiting. This again does not seem like an issue at first; maximizing exploitation is generally what we want. In theory, this is true, and the measurement works well, but if we do not have enough variation in requirements between tested instances, this may present skewed results.

Developing arms to test with that are very even in performance is a difficult and time-consuming process. If too few different instances are used, the resulting arms may not be balanced on other instances, but each added instance increases testing time for every combination. As with many combinatorial problems, this quickly causes the number of combinations to become unreasonably large and thus limits the feasible testing. Papers on the development of MAB algorithms typically evaluate their algorithms by counting the number of times the best arm is pulled compared to the total number of pulls. Such a strategy is not easily applicable here however, since our arms are not just a random number generator with fixed parameters, meaning that we cannot easily know which arm is the best.

The best solution, then, is to look at aggregated results across many problems and instances, across arms at different "distances" and levels of performance, and hope that the variation they provide is enough for the results to give a good picture of the overall performance of the algorithms. Critically, this means that when considering results, only the relative performance of different algorithms is comparable. The arms themselves are not being evaluated, but it is the arms, or the hyperparameter sets they represent, that have the bigger effect on the actual result.

[3]These were determined through a series of much shorter tests, on fewer threads and with much less powerful hardware, and using only a few instances of each problem. Other parameters may be better in the full benchmark environment.

5 Experimental Results

Even with the previously described difficulties of properly evaluating the algorithms' performance, an evaluation must be made. This section demonstrates results across a set of problems that is hopefully varied enough that meaningful conclusions can be drawn. Section 5.1 presents the performances results of the benchmarks with the first set of arms, while section 5.2 does the same for the second set. Section 5.3 then shows some pull-related statistics for both these tests, allowing deeper analysis of the overall performance.

These tests were performed in the same way, on the same problems, instances, and hardware as the results in section 3.5. While it is therefore possible to compare these results to those in section 3.5, it is mostly meaningless to do so, as section 4.5 explains. Despite the fact that the same instances and measurements are used, the evaluation here compares the results of the various algorithms to each other. How well they perform compared to the default annealer used in the earlier tests is in large part dependent on the annealing schedules used in the arms. These performance measurements are used only as a proxy for the much less quantifiable MAB algorithm performance, since a good algorithm should result in better arms being pulled more often, causing a smaller optimality gap than a bad algorithm.

5.1 Benchmarking the Algorithms

Figures 9 through 12 show the results for each of the four problems using the different multi-armed bandit algorithms. These schedules have been chosen primarily due to their similarities, meaning that good performance is not necessarily expected. The intent is only to evaluate the relative performance of the different algorithms compared to each other.

The TSP results are shown in figure 9. These all follow roughly the same curve, but at fairly different rates. Surprisingly, the best-performing algorithm for timeouts of up to a minute is ETC, but Bernoulli-based Thompson sampling overtakes it for longer timeouts, followed by KL-UCB and Poisson-based Thompson sampling for the longest timeouts. OCUCB on the other hand performs quite badly, being the worst algorithm at every timeout.

Figure 10 shows the percent of solved n-queens instances at different timeouts using each of the algorithms. The most noticeable feature of this graph is the ETC line, which is an extreme outlier, far below the others. Bernoulli-based Thompson sampling stands out here as well, with performance closer to ETC than to the other algorithms until the halfway point, at which it suddenly overtakes everything else. Apart from that, OCUCB is now among the top-performing algorithms, being one of only two algorithms to surpass an 80% completion rate within 30 seconds.

Figure 11 plots the results of the TSPTW tests. Similarly to figure 10 the ETC results are a clear outlier. The other methods perform extremely similarly, coming very close to or reaching the same value within 60 seconds.

Figure 12, finally, shows the knapsack test results. These look quite similar to those of the figure 11, with the ETC algorithm being significantly worse than the others at the shorter timeouts, although it nearly catches up to the best algorithms at the longer timeouts. Poisson-based Thompson sampling and KL-UCB both perform comparatively badly here, which is interesting as they both perform well on both TSP and n-queens, with KL-UCB in particular being the second best algorithm overall on both those problems. Here the best algorithm overall continues to be Bernoulli-based Thompson sampling, with OCUCB and plain UCB only slightly behind it. These both perform well on TSPTW as well, but not on n-queens and knapsack.

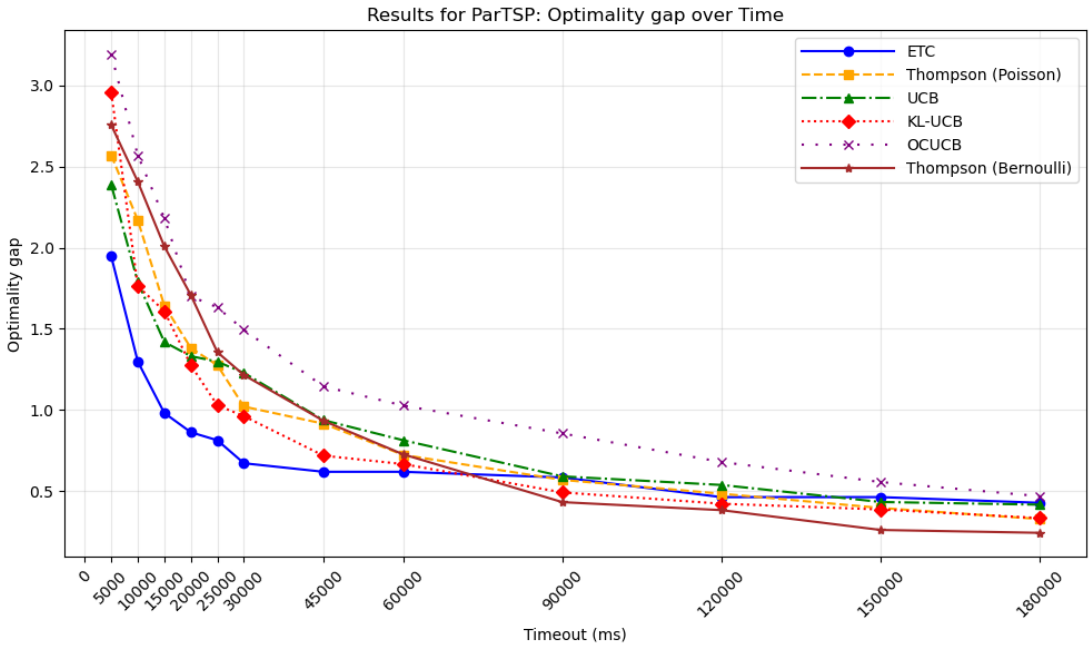


Figure 9: Mean optimality gap of TSP instances at different timeouts. Lower is better.

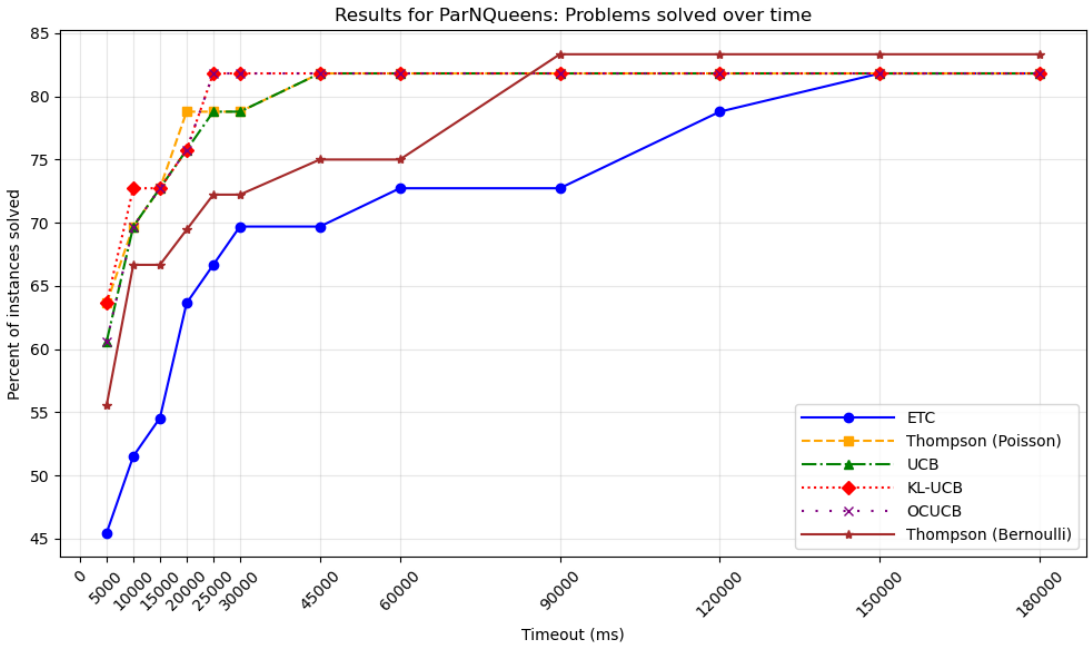


Figure 10: Percent of solved n-queens instances at different timeouts. Higher is better.

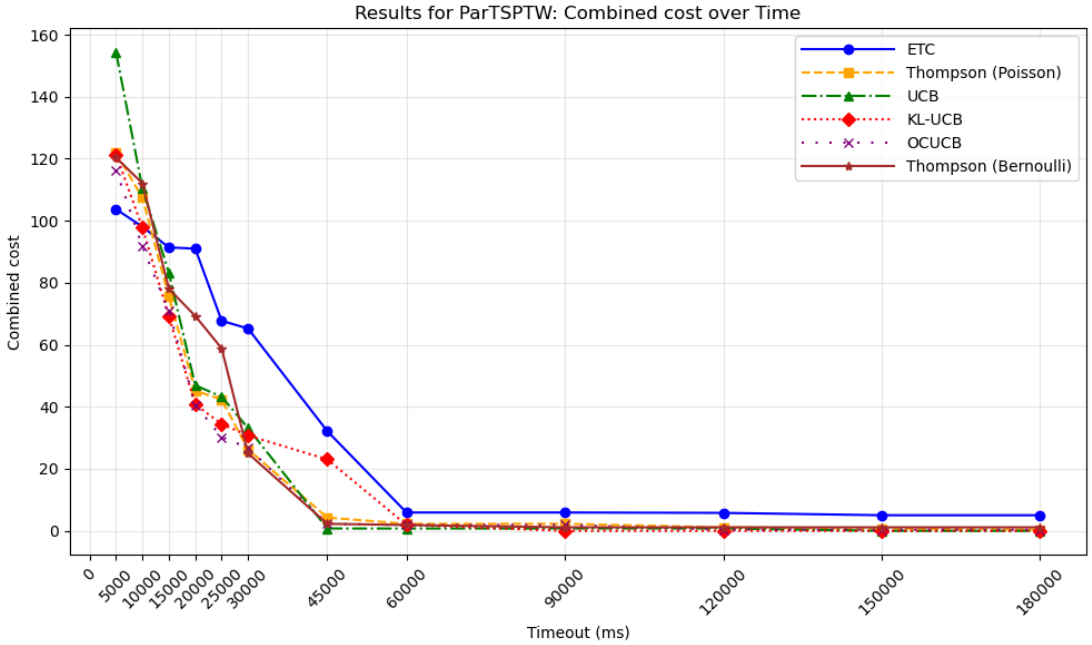


Figure 11: Mean combined cost of TSPTW instances at different timeouts. Lower is better.

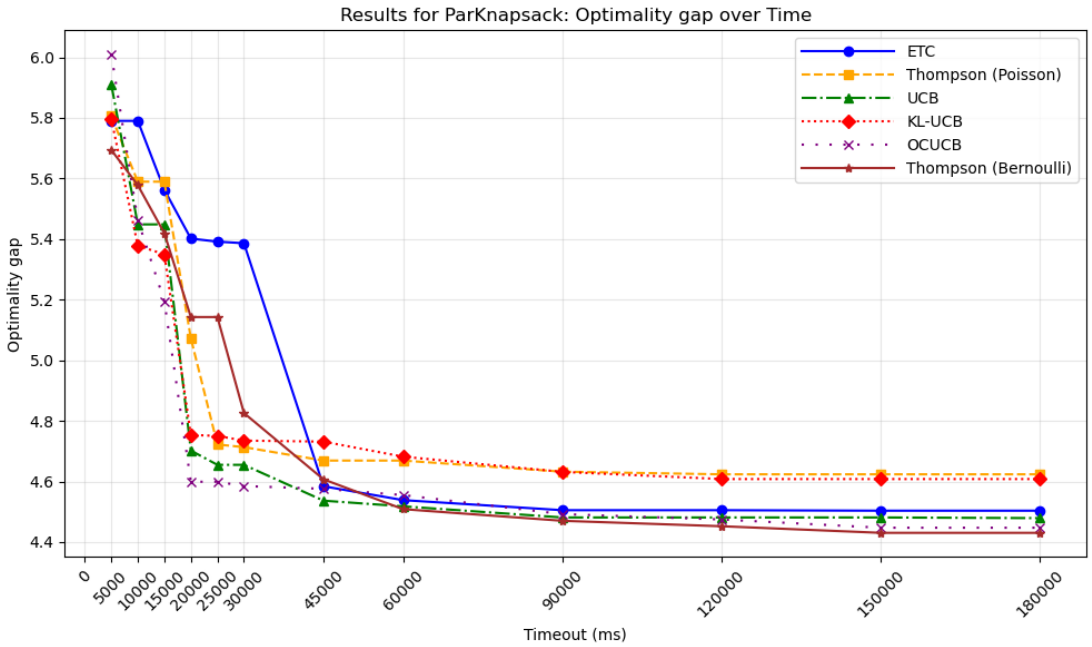


Figure 12: Mean optimality gap of Knapsack instances at different timeouts. Lower is better.

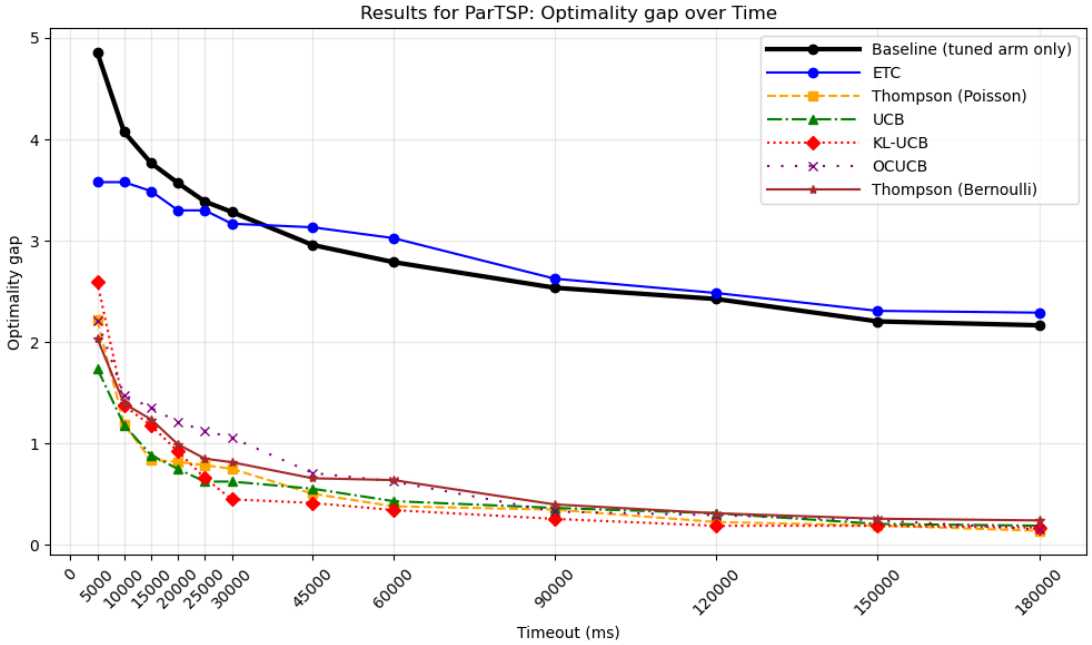


Figure 13: Mean optimality gap of TSP instances at different timeouts. Lower is better.

5.2 Using Tuned Annealing Schedules

Plots 13 through 16 show the results of the same benchmarks using the good arm set. Results are unsurprisingly significantly better than in section 5.1. Each problem has also been benchmarked using only this good parameter set for it. The expected outcome was that these results, which are shown as a baseline, would outperform the MAB results since they never pull the suboptimal arms. If the MAB algorithms perform well, however, this difference should be small.

Throughout these results we see that KL-UCB performs very well, often being the best algorithm, though Bernoulli-based Thompson sampling beats it on TSPTW. Differences between the "good" algorithms are often quite small and their results tend to converge with time. ETC underperforms on all four problems, sometimes by a small margin and quite drastically on other problems. OCUCB is the most inconsistent, performing fairly badly overall, but also rather well in some places, such as the first minute of the knapsack tests. Results are overall very mixed for the short timeouts. This is not unexpected due to the highly random nature of the process, but we also see that results quickly begin to stabilize.

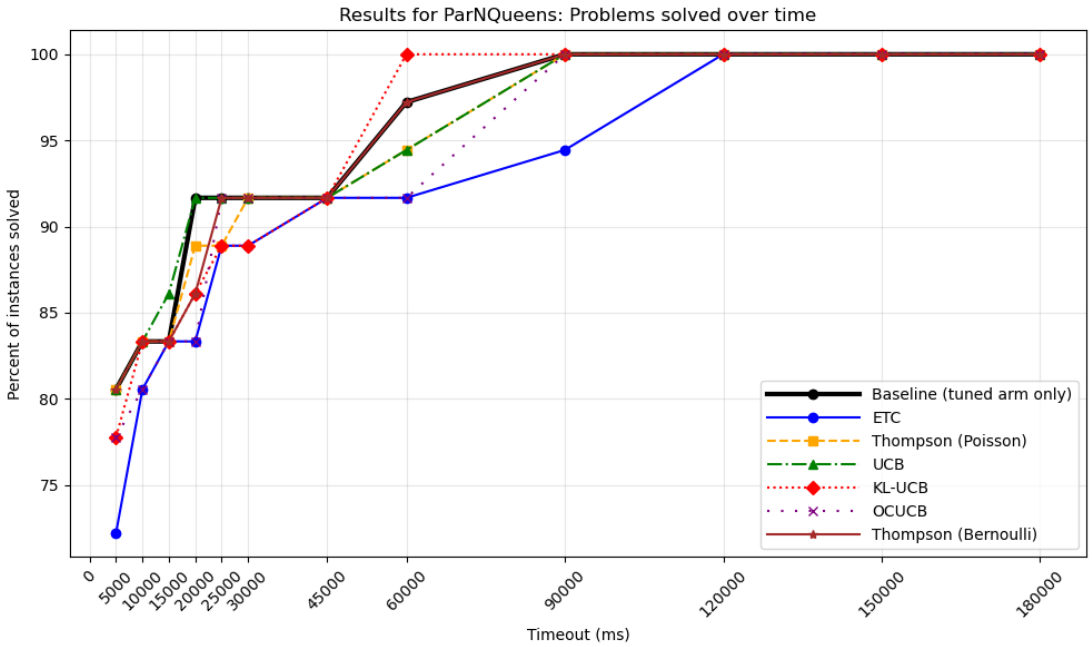


Figure 14: Percent of solved n-queens instances at different timeouts. Higher is better.

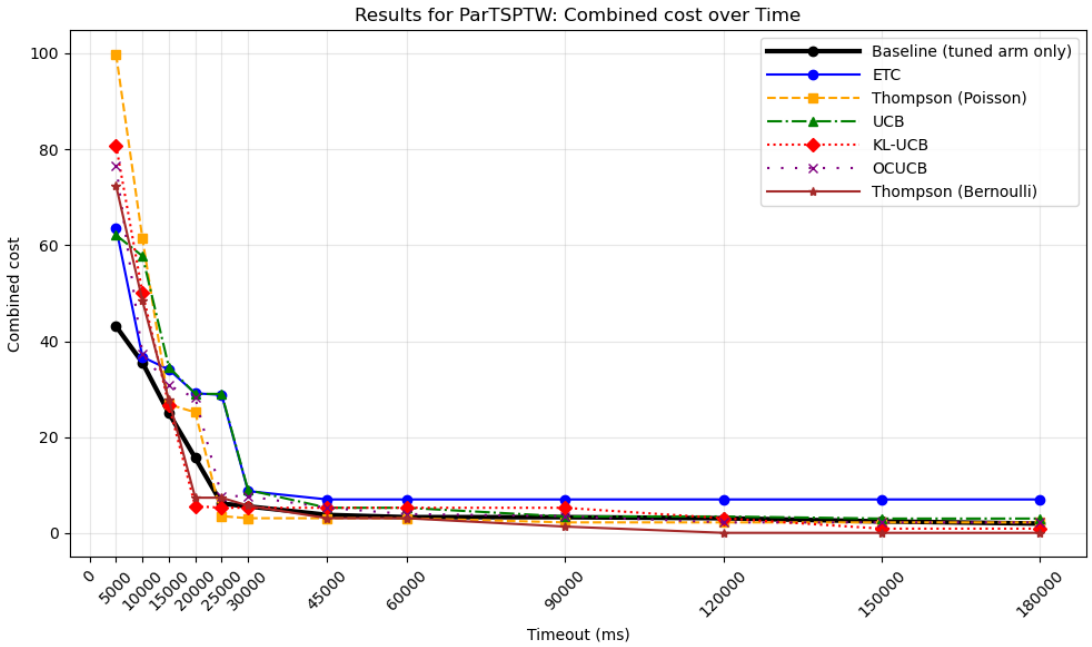


Figure 15: Mean combined cost of TSPTW instances at different timeouts. Lower is better.

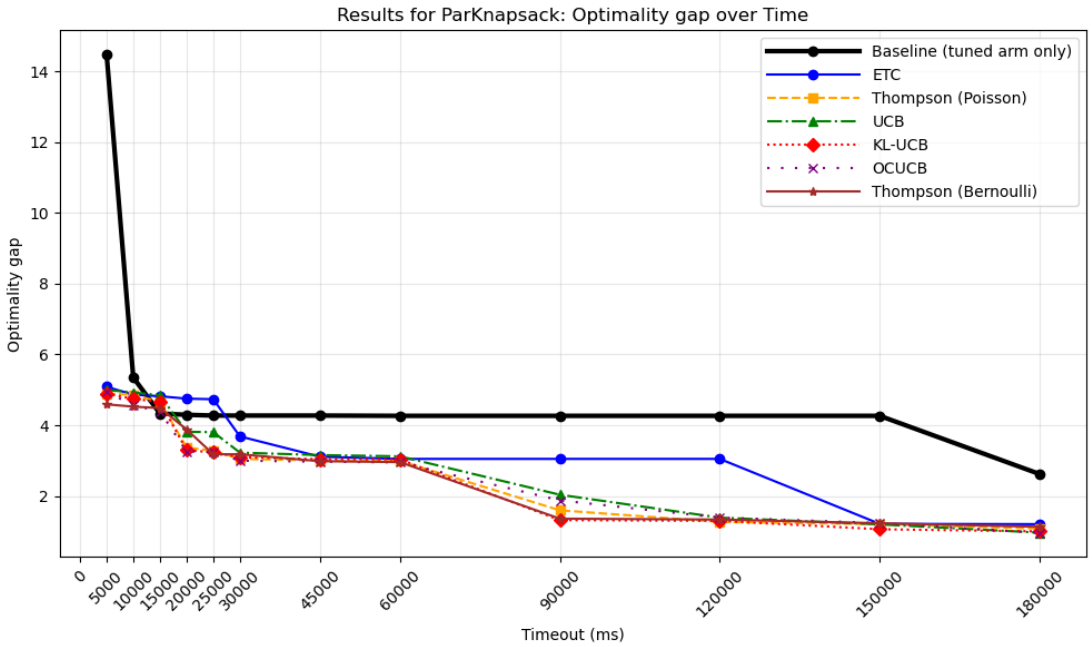


Figure 16: Mean optimality gap of Knapsack instances at different timeouts. Lower is better.

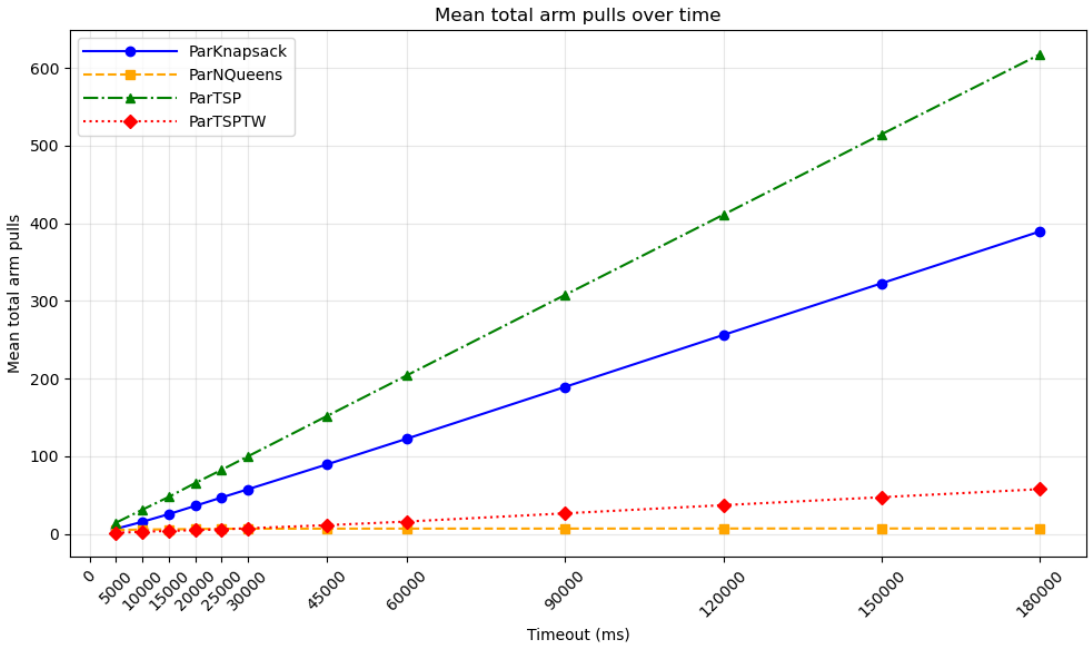


Figure 17: Mean number of pulls of the problems at different timeouts.

5.3 Pulling Arms

In order to be able to properly contextualize these results and analyze them, we also look at some statistics for the number of pulls made in various tests. Figure 17 first shows the mean total number of pulls across the four problems. Tables 6 and 7 then show a further breakdown of pull counts across different arms for some of the benchmarks.

Figure 17 shows the mean total number of pulls recorded at different timeouts across all instances of the four tested problems in the first test. The second test shows similar results, but with recorded pulls for TSP and the knapsack problem increasing even faster, while TSPTW and n-queens results remain similar. The exact number of pulls here is not important; what matters is how it scales. These all scale linearly with time, but at very different rates. Both the knapsack problem and TSP scale quite well, with several hundred or thousand pulls after three minutes. TSPTW and the n-queens problems scale nowhere near as well however.

Tables 6 and 7 show the mean number of recorded pulls of each of the arms in the tests at the three-minute timeout. Since the solver records a pull only when the metaheuristic decides that it is finished, any metaheuristics still running when the solver terminates will not be recorded. These numbers vary widely, from over two thousand to just over two recorded pulls. The reason these numbers are interesting is that they can show how consistent or inconsistent an algorithm is, as well to help explain the algorithms' performance relative to each other.

Table 6 shows, for example, that in the first test the ETC results are quite consistent, while they are far less so in the second test. This can explain why in figure 9 it performs fairly well, while in figure 13 it performs much worse in relation to the other algorithms. In this second test, ETC performs far worse than the others. The table reveals that the algorithm chooses mostly arms 2 and 3, while the more performant methods all prioritize arm 4. Similarly, KL-UCB, which performs very well, is among the most consistent, pulling arms 1, 2, and 3 much less than most of the other algorithms.

In table 7 we again see the strong consistency of KL-UCB and both Thompson sampling algorithms, especially the Bernoulli-based one. Here this seems like more of a problem, however; figure 12 shows KL-UCB and Poisson-based Thompson sampling performing very similarly, as the numbers might suggest, but now being beaten by the other methods. Bernoulli-based Thompson sampling continues to perform well, however. In figure 16 on the other hand, KL-UCB is again the best algorithm.

	Test 1				Test 2			
Bandit Algorithm	Arm 1	Arm 2	Arm 3	Arm 4	Arm 1	Arm 2	Arm 3	Arm 4
ETC	4.00	4.00	589.20	35.87	4.00	2412.00	1576.33	51.20
Thompson (Po)	17.93	15.60	480.13	110.27	14.27	53.40	34.47	877.07
Thompson (Be)	21.80	11.13	433.53	85.73	12.40	45.60	35.13	882.73
UCB	41.20	33.60	308.53	217.27	41.40	140.60	126.60	806.20
KL-UCB	28.27	15.80	413.73	150.60	12.47	55.53	33.20	925.87
OCUCB	56.13	54.40	275.20	235.53	92.40	234.73	214.13	782.40

Table 6: Mean number of pulls recorded on each arm in the TSP tests.

	Test 1				Test 2			
Bandit Algorithm	Arm 1	Arm 2	Arm 3	Arm 4	Arm 1	Arm 2	Arm 3	Arm 4
ETC	123.27	2.40	332.00	2.40	105.13	1134.67	902.00	37.93
Thompson (Po)	55.73	27.60	149.07	150.60	108.27	238.13	440.07	405.80
Thompson (Be)	12.27	17.47	195.60	79.33	5.40	25.40	131.20	678.13
UCB	136.33	110.53	52.27	49.80	95.53	324.93	335.27	410.33
KL-UCB	5.67	2.80	307.67	86.07	6.13	20.87	52.27	735.60
OCUCB	113.07	63.00	93.73	83.13	147.20	275.80	280.07	338.20

Table 7: Mean number of pulls recorded on each arm in the Knapsack tests.

6 Analysis

Just as with the communication strategies in the results of the parallelization testing from section 3, the difference between the MAB algorithms (apart from ETC) is small. This section analyzes the results and discusses possible reasons for why they look as they do. Section 6.1 discusses the details of the implemented algorithms and their results. Section 6.2 then analyzes the statistics for how the different algorithms pull the various arms. What this means for the outcome and project in general is discussed in section 6.3, and section 6.4 finally looks at the defaults and how they should be changed.

6.1 Analysis of the Algorithms

In many of the figures in sections 5.1 and 5.2 the ETC algorithm presents a clear outlier, often performing considerably worse than the other methods. This is likely caused by a small but significant difference in the algorithm's construction. Unlike the other algorithms, the ETC algorithm does not prioritize trying all the arms as fast as possible. It instead pulls only the first arm until the exploration limit is reached, then only the next arm, and so on. One practical effect of this is that the beat-the-mean reward the other algorithms use cannot be used with ETC. It may seem that just changing the pulling order to be more balanced between arms would resolve this issue, but the easy points available in the first few pulls would still skew results too much. The problem is the low number of pulls used in the exploration phase – just four pulls per arm in the benchmarking configuration. Increasing the number however leads to issues for problems and instances that lead to a low number of pulls, since it would cause an exploration vs. exploitation imbalance.

The ETC algorithm is thus forced to use a reward based on the number of improving moves instead of one based on the rolling average of best costs. As previously described, this number should, on average, correlate with performance, but does not have to do so in practice. This could explain why the algorithm so often performs worse than the others; the weaker reward design causes it to choose a suboptimal arm. Differences between ETC and other algorithms are the largest on the n-queens instances. Here the problem is not caused by reward design, but by the fact that restarts rarely happen. Each arm instead lasts a relatively long time, with the solver stopping as soon as a solution is found. Figure 10 shows a much larger differences than 14. This is because of the four arms in the set, the first two, which initially take up all eight threads, are not good enough to find a solution. Once one of these terminates the far better arm three is pulled, leading to a solution being found. Changing the ETC implementation to pull arms in a fairer manner would thus almost certainly lead to improved n-queens benchmark results, but would not affect results on the other problems.

Although it has its issues and often performs badly, ETC does initially outperform the other algorithms in the first TSP tests, as figure 9 shows. It is hard to attribute this to anything other than random chance. Table 6 does show that it is easily the most consistent algorithm on this problem, which could explain that it performs well, but not why it is surpassed for the longer timeouts. Looking more closely at the table, the other algorithms can be seen to also make a significant number of pulls on the fourth arm. A possible explanation is that arm three performs the best on average, but arm four has a wider distribution, allowing it a low chance of outperforming it, and causing algorithms that pull it enough times to actually score better.

The plots reveal ETC as a clear loser, but the same cannot be said when it comes to picking a winner. Disregarding figure 12, the first set of knapsack problem benchmarks, however, KL-UCB has either the best results or is near the best results in each test. In the n-queens tests it seems

to most quickly solve the problems, in the second knapsack problem benchmark set it performs the best, as does it for both sets of TSP benchmarks. In the TSPTW benchmarks it doesn't perform quite as well, but where it is not the best it is at least not far behind. In the first set of knapsack problem benchmarks the results are almost the opposite of most of the other tests. Here the KL-UCB and Poisson-based Thompson sampling have the worst results, with Bernoulli-based Thompson sampling winning.

The same is true for Bernoulli-based Thompson sampling as well, but with the second TSP test (figure 13) being the one case it underperforms, albeit only by a small margin. It also generally seems to perform worse than many of the other algorithms with short timeouts, especially for the first arm set.

It is hardly surprising that KL-UCB outperforms the basic UCB algorithm that it is based on. The OCUCB algorithm however is a newer development, but its strength lies in the strong bounds it provides rather than a practical advantage. It must be remembered that the dynamic reward design means that the bounds can no longer be guaranteed to hold, reducing its utility.

Poisson-based Thompson sampling also consistently performs well, and generally quite similarly to KL-UCB. This may be because both methods are designed to use the results to approximate a distribution, which they in turn use for their arm choices. The odd thing is that although KL-UCB assumes a Bernoulli distribution of rewards, it is the Poisson-based Thompson sampling algorithm that performs similarly to it.

A reason KL-UCB and Thompson sampling specifically perform well is that unlike the other algorithms, they have some knowledge about the expected structure of the rewards encoded in their design. We know from the reward design that it will be binary, and these algorithms are designed to take advantage of this. Unlike the basic asymptotically optimal UCB for example, which knows nothings about the reward until it starts to "see" this pattern in the pulls, they can immediately work to determine the correct distributions. This is likely especially important when few pulls have been recorded; early in the process or for those problems or instances that make very few pulls. The downside is that they are less flexible, so if the reward is replaced with one using a different structure, the algorithms must also be updated to match the new (expected) distribution type.

6.2 Analysis of Pull Counts

The deeper breakdown of pull counts in section 5.3 allows looking at how the algorithms pull different arms. Things like their consistency can provide important insight into the algorithms' performance, as can the differences in which arms they choose. It also allows some surprising conclusions about hyperparameter tuning in general.

Figure 17 shows the drastic differences in scaling between the various problems. Both TSP and the knapsack problem average hundreds of pulls, giving the MAB algorithms ample time to work. TSPTW and the n-queens problem average far fewer pulls however. For the n-queens problem this is not really an issue, as it is just caused by the solver terminating after finding a solution. For TSPTW, on the other hand, this bad scaling is potentially a serious issue. At the three-minute timeout the solver records just 58 pulls on average in the first test and 52 in the second. The bandit algorithms determine the best arm by recording pull results, becoming more likely to pull the best arm over time. If the algorithm is not making new pulls, it is not adapting to results, and thus cannot improve. This means that the algorithm will spend more time pulling bad arms, although if the arms are all at least decent this may still be an improvement over not using multi-armed bandits. The metaheuristic cycles between heating and cooling phases,

and if enough of these cycles happen without the solver making any improvement it will do a full restart, which is when a new arm is selected. Changing metaheuristic parameters while the metaheuristic is running is theoretically possible, but would add significant complexity to both the technical implementation as well as the MAB algorithms' evaluation of arm performance.

The reason for these issues lies in the annealer design. The annealer counts the number of cycles it performs without improvement, with a cycle being considered an improvement if its best cost is better than that of the previous cycle. If the solver then has a bad cycle with a worse best cost than the previous, the next cycle would count as an improvement if it brings the cost back to the best cost of two cycles ago. In a case with such a bad cycle every few cycles, the solver will not restart even though it may not be making any actual improvements either. The solution here is to ensure that the metaheuristic always restarts and does not get stuck. Solving this problem is easier said than done. It may seem like an easy fix to just change the comparison to be against the best cost seen in the current pull and not just in the most recent cycle, but such a change may have unintended consequences. This design is used to allow the solver to make "bad" decisions, and that change would limit its ability to do this, meaning that it cannot be made without thorough investigation.

The figure also shows why it is impossible to fix a good size for the ETC algorithm's exploration phase. The size used in the testing is four pulls per arm, as this should allow the solver to move into the exploitation phase even on problems like TSPTW with lower pull counts and with shorter timeouts. For problems such as TSP on the other hand, where the mean pull count is in the hundreds, it would likely be better with a much longer exploration phase, such as ten or twenty pulls per arm. This would give the algorithm much more time to test different arms and reduce the risk of random chance causing a bad decision.

As previously mentioned, tables 6 and 7 provide some insight into the consistency of the algorithms. KL-UCB and Bernoulli-based Thompson sampling, the overall best-performing algorithms, also appear to be the most consistent, especially throughout the second test. While the other algorithms seem to very quite a bit between arms, KL-UCB strongly prefers arm 4 on both TSP and the knapsack problem, while the other algorithms spread their pulls much more. Even though it seems to perform very badly, the other algorithms all average at least 95 pulls of arm 1 in test 2 of table 7. OCUCB, the least consistent algorithm is terms of performance, also seems to be the algorithm with the widest spread of its pulls. If the algorithm is having trouble picking one arm, or is not consistent with which arm it picks across runs or problem instances, this could explain why the performance seems to vary so much. An even deeper analysis could be carried out here by analyzing the pull counts of each arm across various timeouts, but would also require a significantly expanded results section.

6.3 Analysis of Tuned-Arms and Discussion of results

The second round of benchmarks, utilizing arms tuned specifically for these problems, reveal some very interesting results, with important implications on the use of multi-armed bandits for CBLS.

The three non-default arms used in those benchmarks were tuned for the problems in a process involving many short tests on one instance of each of the problems. Even though several hours were spent on this tuning, all four problems show that the baseline of these tuned arms perform at or below the multi-armed bandit benchmarks. This shows that even though one of the four arms was tuned specifically for each problem while the others were not, using this mix of arms still produces better results.

In table 6 we also see that every algorithm except ETC strongly prefers arm 4. This is very surprising, since arm 2 is the one that was tuned for TSP. It explains why the baseline in figure 13 performs so badly, and why it is similar to the ETC results. These numbers serve to really exemplify the difficulties of hyperparameter tuning and how instance-specific it can be. One of the main motivations for this work was to help mitigate these problems. By allowing the solver to work with several good metaheuristic parameter sets, whether they are tuned for the specific problem or not, the solver is able to do its own tuning, finding the best parameters for each individual instance.

It must unfortunately be stated here that the effects of random variance can still not be discounted entirely. Each problem except n-queens has five instances used in the benchmarks, and each of these is run three times. Every line in figures 9 through 16 thus corresponds to at least 15 runs of the solver. The solver has an incredible amount of random variance to be overcome. A randomly generated solution is probed randomly, with worsening probes being accepted based on a random value. If a Thompson scheduling algorithm is used, these also pick arms to pull by sampling from random distributions meant to approximate those of the arms. Regardless of algorithm, if two arms are judged to be equivalent, one will be chosen at random. 15 runs of the solver then might not be enough to overcome this completely, meaning that the exact rankings of the algorithms, especially when results are close, are of uncertain value.

Even better results could likely be achieved with further tuning to various instances of the same problem, but this seems like a good compromise between manual tuning and solver performance. The idea is that MAB should allow the solver to perform close to where it would be if it were manually tuned for each problem, and these results show that it can do precisely that.

6.4 The Default Annealing Schedule

A side effect of this benchmarking with various arms as well as the process of hyperparameter tuning for the second benchmark set is some insight into the performance of the default annealing schedule. This schedule was used as one of the arms in both tests, and both times the pull counts such as those shown in tables 6 and 7 reveal that it is rarely chosen. Assuming the MAB algorithms work as they should, pulling whichever arm seems the most promising, this indicates that the default schedule does not work very well. Further reinforcing this suspicion is the fact that the parameters of the three tuned arms in the second benchmark are all quite similar, indicating that a good default would be somewhere in that area. The default schedule's minimum uphill acceptance ratio γ of 0.75 is especially far from the others, which use 0.3 or 0.2. Combined with its fairly low heating rate α_h of 1.2, this means that it will spend far longer in the heating phase, cumulatively making many more bad moves. These factors both affect how far from its current minima the solver will move in each heating phase. Spending as much time at such high temperatures as the default schedule does, it can likely move quite far through the search space in each heating phase. Further investigation is necessary before it can be said that this is true for this schedule specifically, but too many negative moves will cause the solver to perform what could be called a pseudo-restart, changing a large amount of the variables to more or less random values.

The high heat here could also potentially explain the similarities between communication strategies seen in section 3.5. Beam search would be expected to perform quite differently from fully parallel search, but as the figures show, it is actually very similar. If the amount of heat over time applied in the annealing schedule is high enough, the number of random moves made in the initial heating phase after each restart will cause it to randomize its variable values.

7 Conclusion

The work in this thesis has significantly improved the solver Atlantis' performance by identifying and resolving several critical bugs. Furthermore, it has shown that the use of multi-armed bandit algorithms for automatic hyperparameter tuning is able to improve performance even further, while simultaneously reducing user workload. A final summary and conclusion of this work is presented in section 7.1. Even with all the work that has been done during this thesis however, there is a large amount of work that remains to be done in the future. This comes both from things that have not been done due to limits in time and scope, as well as from new ideas that have arisen during the development process and from the results. Section 7.2 discusses that work.

7.1 Conclusion

In this thesis the aim was to accomplish two things. First, to investigate and possibly improve previously measured performance issue in the CBLS solver Atlantis, and then to examine the use of multi-armed bandits for metaheuristic selection and hyperparameter tuning. Both parts have resulted in significant improvements to the solver's performance.

In the first part of this thesis, Atlantis' bad performance in general and bad parallel performance specifically were investigated. This lead to several critical bugs being resolved. Benchmarking shows that the solver does indeed parallelize well, and any other result would have been extremely surprising. The different inter-thread communication strategies the solver can use unexpectedly proved to have very little effect on performance, however. Finally, the big constraint on parallelization was shown to be cache contention, as had been theorized already during the parallelization process.

The solver was extended to allow many sets of metaheuristic parameters to be supplied at once, and to easily be able to use different MAB algorithms and rewards. Six algorithms were implemented and thoroughly benchmarked, with Bernoulli-based Thompson sampling and KL-UCB performing the best, although differences were mostly very small. Results show that regardless of whether hyperparameters are tuned or not, use of multi-armed bandits is beneficial to the solver's performance.

The combined work in this thesis has resulted in Atlantis' performance improving by several orders of magnitude. The multi-armed bandit implementation is especially important, because it shows that in the future performance with manual hyperparameter tuning can be approached by, or even surpassed by, some predefined set of arms. This work continues to bring the solver closer to the ideal of a solver with the performance of local search but where the user only has to model their problem rather than perform any manual work designing and tuning the solver.

7.2 Future Work

The most important tests have been performed, but many aspects remain untested. The implementation allows any number of arms to be used, but both sets of benchmarks were performed using four arms. It would therefore be interesting to investigate the optimal number of arms. Such testing would need to be combined with further testing into what level of tuning should be put into the arms, and would preferably be performed on a broader set of problems. The first of these topics has not been investigated due to the time constraint on this work, while the second is one that came up in the testing process.

This extension allows the use of multiple parameter sets, and has further shown that using them is beneficial for the solver's performance, but the default option remains the single default

annealing schedule. Replacing this with some new set of default arms, or at the very least a better default schedule, seems like a clear improvement to the solver. Once a sensible set of hyperparameters has been developed, such a change would likely be very simple.

Six different MAB algorithms have been implemented, but there are many other options that could be considered. When it comes to reward design, there are countless different dynamic reward options using various means and methods of normalization that could be developed. Both the MAB algorithm and rewards have been designed in a modular way, allowing new versions of both to be easily implemented. Further testing of different algorithms and rewards could hopefully further improve performance.

The tests were carried out using different annealing schedules for the preexisting simulated annealing metaheuristic, but the MAB does not have to be limited to this. If other metaheuristics such as taboo search were to be implemented, these could be used alongside simulated annealing in the multi-armed bandit, allowing the solver to freely pick not just metaheuristic parameters, but the entire metaheuristics. Similarly, the simulated annealing implementation could be further modularized, allowing the heating and cooling phases to be tuned separately for example, or letting different settings be used based on where in the search process the solver is, having one parameter set to quickly find good regions of the search space and another when near a local optima.

A change which could have certain benefits, but also some significant drawbacks, is to allow the metaheuristic to trigger restarts if an arm runs for too long. This could improve performance for problems like TSPTW, where arms last a long time, reducing the MAB algorithm's ability to pick a good arm. The risk with such an approach is that the solver could prevent optima from being found by restarting before they are reached. Due to this risk, further testing is necessary to determine if such a change would be beneficial.

Finally, all benchmarking of these improvements have been carried out using fully parallel search, meaning that every pull of an arm works completely independently, with the only inter-thread communication being when results are reported to the thread controller and arm selectors. The solver has support for two other levels of inter-thread communication, and it would be interesting to see how multi-armed bandit performs with these. These could also be further integrated with the bandit design to allow the solver to also choose communication strategies alongside the metaheuristics. Having some threads working with fully parallel search and exploring the search space, while others beam-search to really dive towards the optima could potentially perform very well.

References

[1] Behnaz Aghaabdollahian, Babak Javadi, and Adel Aazami. "An overview of vehicle routing problems: A comprehensive classification and description of types". In: *Transportation Research Interdisciplinary Perspectives* 36 (2026), p. 101808. ISSN: 2590-1982. DOI: 10.1016/j.trip.2025.101808. URL: https://www.sciencedirect.com/science/article/pii/S2590198225004877.

[2] I. Zakir Ahmed, Hamid Sadjadpour, and Shahram Yousefi. "Constrained Resource Allocation Problems in Communications: An Information-assisted Approach". In: *MILCOM 2021 - 2021 IEEE Military Communications Conference (MILCOM)*. 2021, pp. 243–248. DOI: 10.1109/MILCOM52596.2021.9652917.

[3] Carolina P. Almeida et al. "Hyper-heuristics using multi-armed bandit models for multi-objective optimization". In: *Applied Soft Computing* 95 (2020), p. 106520. ISSN: 1568-4946. DOI: 10.1016/j.asoc.2020.106520. URL: https://www.sciencedirect.com/science/article/pii/S1568494620304592.

[4] Armin Biere et al., eds. *Handbook of Satisfiability*. 2nd ed. IOS Press, 2021, p. 1484. ISBN: 978-1-64368-160-3. URL: https://www.iospress.com/catalog/books/handbook-of-satisfiability-2 (visited on 04/23/2026).

[5] Gustav Björdal et al. "A constraint-based local search backend for MiniZinc". In: *Constraints* 20.3 (July 2015), pp. 325–345. ISSN: 1572-9354. DOI: 10.1007/s10601-015-9184-z. URL: https://doi.org/10.1007/s10601-015-9184-z.

[6] Leonardo De Moura and Nikolaj Bjørner. "Satisfiability modulo theories: introduction and applications". In: *Commun. ACM* 54.9 (Sept. 2011), pp. 69–77. ISSN: 0001-0782. DOI: 10.1145/1995376.1995394. URL: https://doi.org/10.1145/1995376.1995394.

[7] Daniel Delahaye, Supatcha Chaimatanan, and Marcel Mongeau. "Simulated annealing: From basics to applications". In: *Handbook of Metaheuristics*. Ed. by Michel Gendreau and Jean-Yves Potvin. Vol. 272. International Series in Operations Research & Management Science (ISOR). Springer, 2019, 1–35.ISBN 978-3-319-91085-7. DOI: 10.1007/978-3-319-91086-4_1. URL: https://enac.hal.science/hal-01887543.

[8] Maarten Flippo. "Contributions to a New CBLS Backend for MiniZinc". Available at https://uu.diva-portal.org/smash/get/diva2:1675497/FULLTEXT01.pdf. Master's thesis. Uppsala University, June 2022.

[9] Merrill M. Flood. "The Traveling-Salesman Problem". In: *Operations Research* 4.1 (1956), pp. 61–75. ISSN: 0030364X, 15265463. URL: http://www.jstor.org/stable/167517 (visited on 04/28/2026).

[10] Aurélien Garivier and Olivier Cappé. "The KL-UCB Algorithm for Bounded Stochastic Bandits and Beyond". In: *Proceedings of the 24th Annual Conference on Learning Theory*. Ed. by Sham M. Kakade and Ulrike von Luxburg. Vol. 19. Proceedings of Machine Learning Research. Budapest, Hungary: PMLR, Sept. 2011, pp. 359–376. URL: https://proceedings.mlr.press/v19/garivier11a.html.

[11] Google LLC. *Google Benchmark*. Version 1.9.4. Accessed: 2026-04-01. Google, May 19, 2025. URL: https://github.com/google/benchmark.

[12] Gurobi Optimization, LLC. *Gurobi Optimizer Reference Manual*. https://www.gurobi.com/. Accessed: 2026-06-03. 2026.

[13] Holger H. Hoos and Thomas Stützle. *Stochastic Local Search*. Morgan Kaufmann, 2005. ISBN: 978-1-55860-872-6. URL: https://www.sciencedirect.com/book/9781558608726/stochastic-local-search (visited on 04/23/2026).

[14] Zhao-hong Jia et al. "A combinatorial algorithm for three dimensional multi-container loading problem with different capacity constraints". In: *Computers & Industrial Engineering* 213 (2026), p. 111779. ISSN: 0360-8352. DOI: 10.1016/j.cie.2025.111779. URL: https://www.sciencedirect.com/science/article/pii/S0360835225009258.

[15] Hans Kellerer, Ulrich Pferschy, and David Pisinger. "Introduction". In: *Knapsack Problems*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2004, pp. 1–14. ISBN: 978-3-540-24777-7. DOI: 10.1007/978-3-540-24777-7_1. URL: https://doi.org/10.1007/978-3-540-24777-7_1.

[16] Tor Lattimore. "Optimally Confident UCB: Improved Regret for Finite-Armed Bandits". Version 3. In: *arXiv preprint arXiv:1507.07880* (Feb. 24, 2016). arXiv: 1507.07880 [cs.LG]. URL: https://arxiv.org/abs/1507.07880.

[17] Tor Lattimore. "Regret Analysis of the Anytime Optimally Confident UCB Algorithm". Version 2. In: *arXiv preprint arXiv:1603.08661* (May 6, 2016). arXiv: 1603.08661 [cs.LG]. URL: https://arxiv.org/abs/1603.08661.

[18] Tor Lattimore and Csaba Szepesvári. *Bandit Algorithms*. Cambridge University Press, 2020. ISBN: 9781108571401. DOI: 10.1017/9781108571401.

[19] Frej Lewander, Pierre Flener, and Justin Pearson. "Invariant Graph Propagation in Constraint-Based Local Search". In: *Journal of Artificial Intelligence Research* 83 (July 2025). DOI: 10.1613/jair.1.17482.

[20] Stine Lomborg, Anne Kaun, and Sne Scott Hansen. "Automated decision-making: Toward a people-centred approach". In: *Sociology Compass* 17.4 (2023). DOI: 10.1111/soc4.13097. URL: https://compass.onlinelibrary.wiley.com/doi/10.1111/soc4.13097.

[21] Manuel López-Ibáñez et al. "The travelling salesman problem with time windows: Adapting algorithms from travel-time to makespan optimization". In: *Applied Soft Computing* 13.9 (2013), pp. 3806–3815. ISSN: 1568-4946. DOI: 10.1016/j.asoc.2013.05.009. URL: https://www.sciencedirect.com/science/article/pii/S1568494613001658.

[22] Yannick Molinghen et al. "Reinforcement Learning Methods for Neighborhood Selection in Local Search". In: (Jan. 2026). Accepted at ICORES 2026. DOI: 10.48550/arXiv.2601.07948. arXiv: 2601.07948 [cs.LG]. URL: https://arxiv.org/abs/2601.07948 (visited on 04/23/2026).

[23] Kameng Nip and Peng Xie. "Computational complexity and algorithms for two scheduling problems under linear constraints". In: *Journal of Combinatorial Optimization* 47.4 (Apr. 14, 2024), p. 55. ISSN: 1573-2886. DOI: 10.1007/s10878-024-01122-0. URL: https://doi.org/10.1007/s10878-024-01122-0.

[24] *perf: Performance analysis tools for Linux*. Accessed: 2026-03-06. Linux. 2026. URL: https://man7.org/linux/man-pages/man1/perf.1.html.

[25] Francesca Rossi, Peter van Beek, and Toby Walsh. *Handbook of Constraint Programming*. Elsevier, 2006. ISBN: 978-0-08-046380-3. DOI: 10.5555/2843512.

[26] Olof Rudbeck. *Atlantica. sive Mannheim vera Japheti posterorum sedes ac patria*. Vol. 1–4. Original Latin edition; later editions and translations may vary. Uppsala: Henricus Curio, 1679.

[27] Christian Schulte et al. “Gecode: Generic Constraint Development Environment”. In: *Proceedings of the 2nd International Conference on Integration of AI and OR Techniques in Constraint Programming for Combinatorial Optimization Problems (CPAIOR 2005)*. Vol. 3524. Lecture Notes in Computer Science. Springer, 2005, pp. 207–221. DOI: 10.1007/11493909_16.

[28] Peter J. Stuckey, Kim Marriott, and Guido Tack. *The MiniZinc Handbook*. Accessed: 2026-02-19. MiniZinc Development Team. 2026. URL: https://docs.minizinc.dev/en/stable.

[29] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2nd ed. The MIT Press, 2018. ISBN: 978-0-262-03924-6. URL: https://mitpress.mit.edu/books/reinforcement-learning-second-edition.

[30] William R. Thompson. “On the Likelihood that One Unknown Probability Exceeds Another in View of the Evidence of Two Samples”. In: *Biometrika* 25.3/4 (1933), pp. 285–294. ISSN: 00063444. URL: http://www.jstor.org/stable/2332286.

[31] UPPMAX. *Pelle hardware*. Uppsala University. URL: https://docs.uppmax.uu.se/hardware/clusters/pelle/ (visited on 04/28/2026).

[32] Pascal Van Hentenryck and Laurent Michel. *Constraint-Based Local Search*. The MIT Press, 2005. ISBN: 0262220776.

[33] Jiongzhi Zheng et al. “Integrating multi-armed bandit with local search for MaxSAT”. In: *Artificial Intelligence* 338 (2025), p. 104242. ISSN: 0004-3702. DOI: 10.1016/j.artint.2024.104242. URL: https://www.sciencedirect.com/science/article/pii/S0004370224001784.